

MiNT: Multi-Network Training for Transfer Learning on Temporal Graphs

Kiarash Shamsi^{*1} Tran Gia Bao Ngo^{*1} Razieh Shirzadkhani^{*23} Shenyang Huang²³ Farimah Poursafaei²³
 Poupak Azad¹ Reihaneh Rabbany²³⁴ Baris Coskunuzer⁵ Guillaume Rabusseau²⁶ Cuneyt Gurcan Akcora⁷

Abstract

Temporal Graph Learning (TGL) has become a robust framework for discovering patterns in dynamic networks and predicting future interactions. While existing research has largely concentrated on learning from individual networks, this study explores the potential of learning from multiple temporal networks and its ability to transfer to unobserved networks. To achieve this, we introduce **Temporal Multi-network Training (MiNT)**, a novel pre-training approach that learns from multiple temporal networks. With a novel collection of 84 temporal transaction networks, we pre-train TGL models on up to 64 networks and assess their transferability to 20 unseen networks. Remarkably, MiNT achieves state-of-the-art results in zero-shot inference, surpassing models individually trained on each network. Our findings further demonstrate that increasing the number of pre-training networks significantly improves transfer performance. This work lays the groundwork for developing Temporal Graph Foundation Models, highlighting the significant potential of multi-network pre-training in TGL. Our code is available at <https://anonymous.4open.science/r/MiNT>

1. Introduction

Temporal graph learning has gained significant attention for its ability to model dynamic networks with evolving relationships, effectively addressing challenges that static graph representations cannot overcome (Longa et al., 2023).

^{*}Equal contribution ¹Department of Computer Science, University of Manitoba ²Mila - Quebec AI Institute ³School of Computer Science, McGill University ⁴CIFAR AI Chair ⁵University of Texas at Dallas ⁶DIRO, Université de Montréal ⁷AI Initiative - University of Central Florida. Correspondence to: Kiarash Shamsi <shamsik1@myumanitoba.ca>, Cuneyt Akcora <cuneyt.akcora@ucf.edu>.

Inspired by the remarkable success of pre-trained models in natural language processing (NLP) (Bubeck et al., 2023; Brown et al., 2020; Rasul et al., 2024) and computer vision (CV) (Radford et al., 2021; Awais et al., 2023), there has been a growing interest to develop Graph Foundational Models (GFM) (Mao et al., 2024). These models employ a *pre-train and transfer* strategy, enabling Graph Neural Networks (GNNs) to leverage large-scale datasets for pre-training and efficiently transfer to new tasks with minimal supervision (Wang et al., 2024; Beaini et al., 2023). However, in temporal graph learning, existing literature focuses on training and testing on a single temporal network (Huang et al., 2023; Rossi et al., 2020; Pareja et al., 2020), therefore learning the evolution of only one network. Inspired by the success of large pre-trained models, we explore two fundamental questions about temporal graph models: (1) Can temporal graph models benefit from learning across multiple networks? (2) Can temporal graph models effectively transfer the learned knowledge from pre-training datasets to unseen networks within the same domain?

To investigate these questions, we selected Ethereum-based cryptocurrency networks as a testbed and constructed a novel collection of 84 temporal transaction networks, comprising 3 million nodes and 19 million edges. This dataset facilitates the study of transferability and multi-network training, providing a robust foundation for future research in temporal graph learning. Next, we propose the first algorithm to pre-train temporal graph neural networks (TGNNs) across multiple networks, developing a novel approach named **Temporal Multi-network Training (MiNT)**. By training MiNT on up to 64 cryptocurrency networks, we assess its transfer learning performance compared to state-of-the-art TGNN models. Figure 1 shows how our multi-network models, compared to two SOTA models, namely HTGN and GCLSTM, perform as we train them for graph property prediction tasks. The average performance of MiNT on twenty unseen token networks improves as more training data is provided, outperforming single models trained on individual networks. Our results demonstrate that MiNT consistently performs on par with or even better individually trained models, underscoring its transformative potential to drive

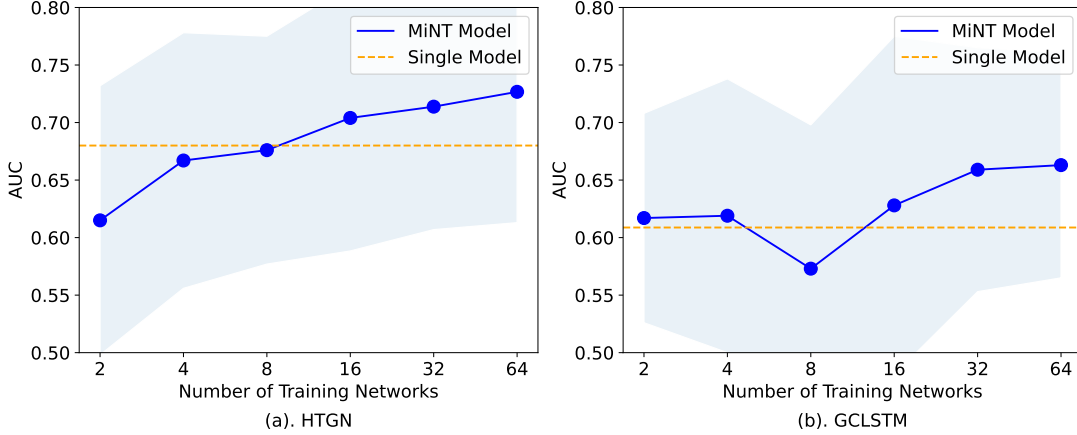


Figure 1: **Scaling behavior of MiNT on unseen networks.** Zero-shot inference performance of MiNT (*Multi-Network Model*) on unseen networks, compared with standard training of individual networks (*Single Model*). The base TGNN models are (a) HTGN, and (b) GC-LSTM. A single model is trained and tested on each test network, while MiNT performs zero-shot inference on each test network. The metric is the average ROC AUC over 20 test networks.

progress in transfer learning for temporal graph learning. Our contributions are:

- **Multi-network Training Algorithm:** We introduce *MiNT-train*, the first algorithm to train TGNNs across multiple networks simultaneously, expanding possibilities for temporal graph learning.
- **Positive Dataset Scaling:** Our MiNT model demonstrates significant improvement as it learns from an increasing number of networks (up to 64), highlighting the advantages of multi-network learning on temporal graphs.
- **Superior Transferability:** MiNT achieves competitive zero-shot inference on 20 unseen token networks, matching or even outperforming SOTA on 14 out of 20 unseen networks. This demonstrates exceptional generalization and transfer capabilities.
- **Extensive Temporal Network Dataset:** We present a dataset of 84 labeled temporal graphs derived from token transaction networks, designed to support future research on scaling, transferability, and multi-network learning.

Reproducibility. Our code is available on <https://anonymous.4open.science/r/MiNT>. The MiNT datasets are available on [Dropbox](#).

2. Related Work

Temporal Graph Learning. Temporal graph neural networks (TGNNs) have shown promising performance in tasks such as link prediction and node classification. Current literature (Rossi et al., 2020; Yang et al., 2021; Chen et al., 2022) focuses on learning from a single network and partitioning the network into a training set and a test set chronologi-

cally. Thus, the objective is to extract patterns from the observed temporal graph and then predict its evolution in the future. In the inductive settings, TGNNs predict accurately for novel nodes from the *same* network that were not observed in the training set (Xu et al., 2020; Wang et al., 2021). In this work, we propose the first algorithm to effectively learn from multiple temporal graphs and focus on the temporal graph property prediction task. Additionally, we test the transferability of TGNNs on novel networks, unobserved during training.

Kazemi et al. (2020) categorized temporal graphs into Discrete Time Dynamic Graphs (DTDGs) and Continuous Time Dynamic Graphs (CTDGs). In this work, we focus on DTDGs, since the temporal graph property prediction task is more appropriately defined over graph snapshots (Shamsi et al., 2024). DTDG methods process each graph snapshot sequentially and use recurrent modules to learn temporal dependencies. For example, GCLSTM (Chen et al., 2022) stacks a graph CNN for feature extraction and an LSTM cell for temporal reasoning. In addition, leveraging the power of hyperbolic geometry, HTGN maps the temporal graph into hyperbolic space and utilizes hyperbolic graph neural networks and hyperbolic-gated recurrent neural networks to model the evolving dynamics. Another SOTA for temporal graph property prediction is GraphPulse (Shamsi et al., 2024), which leverages topological data analysis tools to extract information from each snapshot. However, all DTDG models are designed to train and test on a single network, while in this work, we explore multi-network training.

Graph Foundation Models. Recently, Xia & Huang (2024) outlined the challenges associated with building a graph foundation model as structural heterogeneity, feature

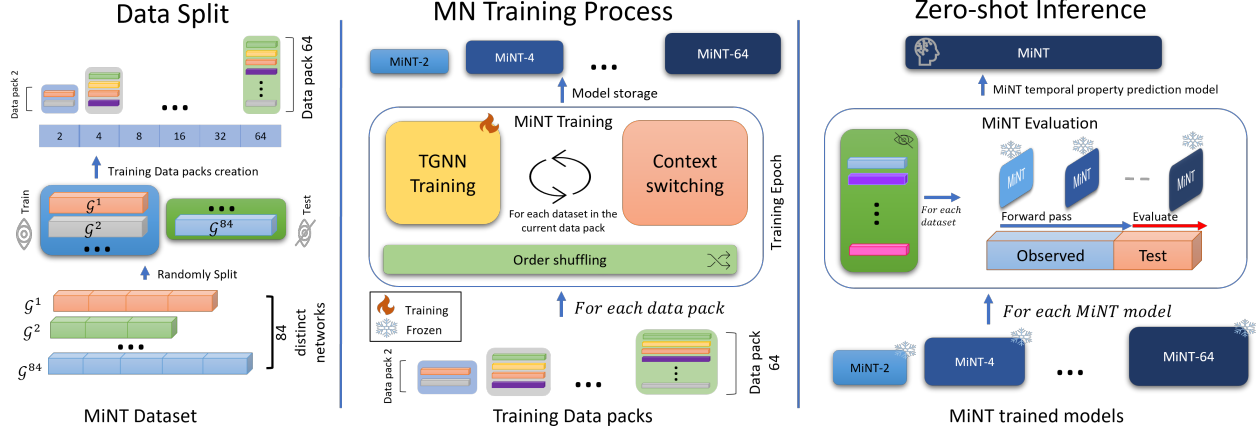


Figure 2: **MiNT framework.** Temporal graphs are preprocessed to generate discrete-time snapshots. Next, the multi-network training pipeline leverages these snapshots to train TGNNs across multiple networks for zero-shot inference on unseen temporal networks.

heterogeneity, fast adaptation and achieving neural scaling law. Neural scaling law is often used to categorize the relationship between model performance and factors in model training and design such as the number of parameters, the size of the training set and the amount of computation required (Rosenfeld et al., 2020; Kaplan et al., 2020; Abnar et al., 2022). Liu et al. (2024) investigated neural scaling laws for static graphs by observing the performance of GNNs given the increase in the model size and training set size. Graph foundation models such as GraphAny (Zhao et al., 2024) and AnyGraph (Xia & Huang, 2024) focus on designing architectures that allow for easy adaption to unseen networks. There are also approaches to build domain-specific foundation models such as those for molecular graphs (Beaini et al., 2023; Sypetkowski et al., 2024; Klaser et al., 2024). In this work, we provide a novel collection of temporal graphs, necessary for building future TG foundation models while demonstrating the significant benefit of pre-training TGNNs on multiple networks for transferability to unseen networks.

Zero-shot Inference. Zero-shot learning has emerged as a powerful approach to enable pre-trained models to extrapolate prediction on unseen data from datasets used in the pre-training stage. Inspired by recent advancements in zero-shot learning and the power of pre-trained models in LLM (Rasul et al., 2024) and CV (Stojnic et al., 2024), Wang et al. (2024) introduced TEA-GLM, a novel framework that aligns GNN representations with LLM token embeddings by a linear projector. The incorporation of LLMs and GNNs enables zero-shot inference on unseen graphs. Additionally, Xia et al. (2024) proposed OpenGraph, an initiative promoting transparency, reproducibility, and community-driven advancements in graph representation learning. In this work, our proposed MiNT algorithm allows pre-trained TGNNs to

achieve zero-shot inference without the need for fine-tuning or modifications, relying directly on the inference pass of a frozen pre-trained model.

3. Background

This section provides the background for temporal graph learning and the temporal graph property prediction task.

In this work, we focus on DTDGs which represent the temporal evolution of the graph in discrete graph snapshots.

Definition 1 (Discrete Time Dynamic Graphs). DTDGs represent a network as a sequence of graph snapshots denoted as $\mathcal{G} = \{\mathcal{G}_1, \mathcal{G}_2, \mathcal{G}_3, \dots, \mathcal{G}_n\}$. Each snapshot $\mathcal{G}_t = (\mathcal{V}_t, \mathcal{E}_t, \mathbf{X}_t, \mathbf{Y}_t)$ corresponds to the graph at timestamp t , where $\mathcal{V}_t$ and $\mathcal{E}_t$ are the sets of nodes and edges, respectively. $\mathbf{X}_t$ and $\mathbf{Y}_t$ are the node and edge feature matrices, respectively. A collection of DTDGs is defined as $D = \{\mathcal{G}^1, \mathcal{G}^2, \dots, \mathcal{G}^m\}$, where m is the number of DTDGs.

Temporal Graph Property Prediction. This task focuses on forecasting a specific property of a temporal graph over a future time interval in a DTDG. Formally, given a DTDG $\mathcal{G}$, we define a target time interval $[t_{k+\delta_1}, t_{k+\delta_2}]$, where δ_1 and δ_2 are non-negative integers and $\delta_1 \leq \delta_2$. At a specific time t_k , the objective is to predict the values of the chosen graph property within the specified future interval $[t_{k+\delta_1}, t_{k+\delta_2}]$. This task is particularly valuable for analyzing the scalability and evolution of temporal graphs. For instance, in financial domains, predicting the growth or shrinkage of the network can provide insights into market dynamics and investor behavior. Moreover, leveraging the DTDGs framework for graph property prediction can uncover critical patterns in domains like cryptocurrency token networks (Akcora et al., 2019).

Algorithm 1 Multi-Network Training for Temporal Graphs

```

1: Input: A Temporal Graph Dataset  $D = \{\mathcal{G}^1, \mathcal{G}^2, \dots, \mathcal{G}^m\}$ , where  $\mathcal{G}^i = \{\mathcal{G}_1^i, \mathcal{G}_2^i, \dots, \mathcal{G}_{n_i}^i\}$ ,  $m$  = Number of networks in training, a TGNN model, and a graph property prediction Decoder
2: for each epoch do
3:   // Order shuffling
4:   Shuffle ( $D$ )
5:   for each network  $\mathcal{G}^i \in D$  do
6:     // Context switching
7:     Initialize historical embeddings  $\mathcal{H}_0$  (reset)
8:     for  $t = 1, 2, \dots, n_i$  do
9:       // Train on snapshot  $\mathcal{G}_t^i$ 
10:       $\mathcal{H}_t = \text{TGNN}(\mathcal{G}_t^i, \mathcal{H}_{t-1})$ 
11:       $\hat{y}_t = \text{Decoder}(\mathcal{H}_t)$ 
12:       $\mathcal{L} = \text{Loss}(y_t, \hat{y}_t)$ 
13:      Backpropagation
14:    end for
15:    Evaluate on the validation snapshots of  $\mathcal{G}^i$ 
16:  end for
17:  Average validation results across all datasets to select the best model
18:  Save the best model for inference
19: end for

```

4. MiNT: Temporal Multi-network Training

In this section, we introduce our Temporal Multi-network Training (MiNT) algorithm, an innovative multi-network pre-training framework designed to be applied to any TGNN architecture for DTDG. By leveraging MiNT pre-training, the base TGNN model can now transfer to unseen networks for zero-shot inference. Figure 2 provides an overview of our MiNT framework, illustrating the process from dataset curation to the model pre-training stage, and finally to zero-shot inference on test networks.

4.1. Multi-network Training

Existing temporal graph learning models typically train on a single temporal graph, limiting their ability to capture similar behaviors and generalize across different networks (Rossi et al., 2020; Yang et al., 2021). In contrast, we consider a classical learning scenario where a training dataset of m temporal graphs $D = \{\mathcal{G}^1, \mathcal{G}^2, \dots, \mathcal{G}^m\}$ is drawn identically and independently (IID) from an unknown distribution, and the learned model is evaluated on a test set of unseen temporal networks drawn from the same distribution.

Our MiNT algorithm trains across multiple temporal graphs by modifying a SOTA single network training model with two crucial steps: *shuffling* and *context switching*. As explained below, these steps render the algorithm network-agnostic, capable of learning from various temporal graphs

to generalize effectively to unseen networks. Algorithm 1 shows MiNT-train approach in detail. As the first step, we load the list of temporal graphs D , where each temporal graph $\mathcal{G}^i$ is represented as a sequence of snapshot $\{\mathcal{G}_1^i, \mathcal{G}_2^i, \dots, \mathcal{G}_{n_i}^i\}$. For each epoch, we shuffle the orders of the list of datasets D to preserve the IID assumption of neural network training.

Order Shuffling. Since previous works focus on training models on a single network for temporal tasks, we incorporate a shuffling step at each epoch to facilitate training on multiple networks and enable inference on unseen networks. The randomized ordering of networks during training at each epoch is important because it helps prevent the model from learning spurious correlations that could arise if the data were presented in a fixed order. Shuffling the datasets promotes randomness in the training process, contributing to more robust and generalizable model performance. Sequentially, for each dataset $\mathcal{G}^i$, we first initialize the historical embeddings, then train the model end-to-end on each dataset $\mathcal{G}^i$ in a similar manner of training a single model, and evaluate the performance on the corresponding validation set of dataset $\mathcal{G}^i$. After training on m datasets from D , we compute the average validation results across these datasets. This average is used to select the best model, which is then used for inference. Early stopping is applied if needed. The importance of order shuffling is demonstrated in the ablation study presented in Table 3.

Context Switching. Many TGNNs store and utilize node embeddings or latent states from previous timestamps at later timestamps; we refer to those embeddings as *historical embeddings* (Yang et al., 2021; Chen et al., 2022; Pareja et al., 2020). In Algorithm 1, this is represented in line 10 as

$$\mathcal{H}_t = \text{TGNN}(\mathcal{G}_t^i, \mathcal{H}_{t-1}),$$

indicating that at time steps t , the temporal graph model takes as input both the current snapshot $\mathcal{G}_t^i$ and the *latent state* $\mathcal{H}_{t-1}$ from the previous time step (similar to RNNs). Resetting historical embeddings at the beginning of each epoch (line 7 of Algorithm 1) is a key step in training a temporal model across multiple networks for several reasons. First, it helps prevent the model from carrying over biases or assumptions from one network to another, ensuring that it can adapt effectively to the unique characteristics of each network. Starting with fresh historical embeddings at the beginning of each epoch enables the model to learn the most relevant and up-to-date information from the current network, improving performance and generalization across different networks. This is equivalent to resetting the initial vector of recurrent neural networks at the beginning of each sequence.

Zero-shot Inference. To evaluate the transferability of each multi-network model, we test the model on the test sets that

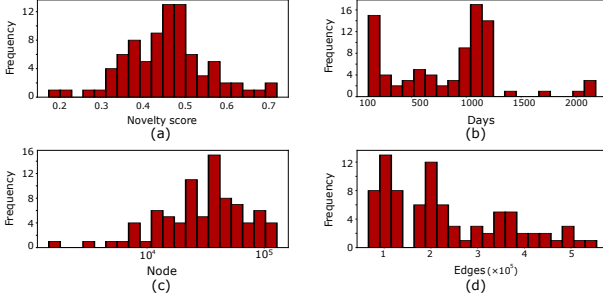


Figure 3: **Network statistics of MiNT networks.** (a) Novelty score, (b) number of days, (c) number of nodes, and (d) number of edges.

are unseen by the models during the training phase. We first divide our networks into two disjoint sets, where one set is used for training obtained by randomly selecting 64 token networks, and the remaining 20 token networks are used to evaluate the performance. In the inference phase, we begin by loading all the weights of multi-network models, including the pre-trained encoder and decoder parameters, while initializing fresh historical embeddings. Then, we perform a single forward pass over the train and validation split to adapt the historical embeddings specific to the testing dataset.

4.2. MiNT Datasets

We introduce a diverse collection of large token networks derived from the real-world Ethereum blockchain (Wood et al., 2014), capturing authentic transaction patterns from 2017 to 2023. The dataset includes 84 distinct ERC20 token networks, each reflecting unique investor and transaction behaviors. Importantly, these token networks operate independently, with varying start dates, durations, and evolution, making them valuable for analyzing shared and unique characteristics within temporal graph structures. The data extraction process and its structure are provided in Appendix A. We split the 84 token networks into two sets: 64 networks are used for training, while the remaining 20 networks, which are never seen during training, are reserved exclusively for zero-shot inference. This split ensures that the model’s ability to generalize to entirely new networks can be effectively evaluated. MiNT contains a diverse set of dynamic graphs in terms of nodes, edges, and timestamps, which are shown in Figure 3. Details on statistics are given in Appendix A.2.

5. Experiments

In this section, we conduct empirical experiments to evaluate the transferability of our proposed MiNT approach to unseen test networks. Our experiments focus on the temporal graph property prediction task for predicting network growth or shrinkage (see Section 3). As weekly forecasts

are common in the financial context for facilitating financial decisions (Kim et al., 2021), we set $\delta_1 = 3$ and $\delta_2 = 10$ for the temporal graph property prediction task, thus predicting the temporal graph property over weekly snapshots. In addition, to show that MiNT is agnostic to the underlying TGNN architecture, we select two widely-used TGNN models as base architecture, namely HTGN (Yang et al., 2021) and GCLSTM (Chen et al., 2022). Experimental results suggest that HTGN has superior performance overall; thus, we adapt this model for in-depth analysis and ablation studies.¹ We also provide detailed experimental results and findings in Appendix F.

5.1. Contenders And Baselines

For comparison, we include heuristics, single-network models, and our pre-trained MiNT models. We refer to models that are trained on only a single network (such as in existing literature) as *single models*. For each temporal graph, we adopt a 70% – 15% – 15% split ratio for the train, validation, and test sets, respectively, and during each epoch, the snapshots from a network are processed sequentially and chronologically by the model.

We train every single model for a minimum of 100 and a maximum of 250 epochs with a learning rate set to 1.5×10^{-3} . We apply early stopping based on the AUC results on the validation set, with patience and tolerance set to 20 and 5×10^{-2} , respectively. We use Binary Cross-Entropy Loss for performance measurement and Adam (Kingma & Ba, 2015) as the optimization algorithm. It is important to note that the graph pooling layer, performance measurement, and optimization algorithm are also shared by the multi-network model training setup.

Persistence Forecast. For our baseline model, we employ a naive approach similar to deterministic heuristics techniques, persistence forecast (Salcedo-Sanz et al., 2022), for label prediction. We further provide details about the persistence forecast baseline in Appendix B. Hyperparameter details are described in Appendix D.

Single Models. We use four models from literature, including HTGN (Yang et al., 2021), GCLSTM (Chen et al., 2022), EvolveGCN (Pareja et al., 2020) and GraphPulse (Shamsi et al., 2024) as our baseline models trained from scratch on individual networks. We further explain each model in Appendix B. To address graph-level tasks, we add an extra graph pooling layer as the final layer. This layer, implemented as a Multi-Layer Perceptron (MLP), takes the mean of all node embeddings, concatenating with four snapshot features at the graph level (i.e., the mean of

¹The default MiNT models refer to HTGN with our MiNT pre-training, for example in Figure 4

Table 1: ROC AUC scores of MiNT models (ours, zero-shot inference on unseen test networks), single models (trained on test networks), and persistence forecasts on test sets across three seeds. Best performance in **bold**, second best underlined.

Network	Standard Training on Individual Networks					MiNT-64 (Transfer)	
	PF	HTGN	GC-LSTM	EvolveGCN	GraphPulse	GC-LSTM	HTGN
WOJAK	0.378	0.479±0.005	0.484±0.000	0.505±0.023	0.467±0.030	0.534±0.020	0.524±0.027
DOGE2.0	0.250	0.590±0.059	0.538±0.000	<u>0.551±0.022</u>	0.384±0.180	<u>0.551±0.022</u>	0.538±0.038
EVERMOON	0.241	0.512±0.023	0.562±0.179	0.451±0.046	<u>0.519±0.130</u>	0.494±0.047	0.517±0.039
QOM	0.334	0.633±0.017	0.612±0.001	0.618±0.002	0.775±0.011	0.618±0.004	<u>0.647±0.019</u>
SDEX	0.423	0.762±0.034	0.720±0.002	<u>0.733±0.028</u>	0.436±0.030	0.723±0.002	0.614±0.020
ETH2x-FLI	0.355	0.610±0.059	0.670±0.009	0.688±0.010	0.666±0.047	<u>0.697±0.009</u>	0.729±0.015
BEPRO	0.393	0.655±0.038	0.632±0.019	0.610±0.012	0.783±0.003	0.746±0.015	<u>0.782±0.003</u>
XCN	0.592	0.668±0.099	0.306±0.092	0.512±0.067	<u>0.821±0.004</u>	0.733±0.003	0.851±0.043
BAG	0.792	0.673±0.227	0.196±0.179	0.329±0.040	0.934±0.020	0.529±0.023	<u>0.931±0.028</u>
TRAC	0.400	0.712±0.071	0.748±0.000	0.748±0.000	<u>0.767±0.001</u>	0.742±0.004	0.785±0.008
DERC	0.353	0.683±0.013	0.703±0.022	0.669±0.009	<u>0.769±0.040</u>	0.696±0.011	0.798±0.027
Metis	0.423	0.715±0.122	0.646±0.023	0.688±0.027	0.812±0.011	0.697±0.013	<u>0.760±0.025</u>
REPV2	0.321	0.760±0.012	0.725±0.014	0.709±0.002	0.830±0.001	0.733±0.019	<u>0.789±0.020</u>
DINO	0.431	0.730±0.195	0.874±0.028	<u>0.868±0.029</u>	0.801±0.020	0.659±0.039	0.779±0.113
HOICHI	0.374	0.807±0.047	0.857±0.000	<u>0.856±0.001</u>	0.714±0.010	0.847±0.005	0.765±0.018
MUTE	0.536	0.649±0.015	0.593±0.030	0.617±0.010	0.779±0.004	0.636±0.003	<u>0.673±0.013</u>
GLM	0.427	<u>0.830±0.029</u>	0.451±0.003	0.501±0.033	0.769±0.018	0.501±0.027	0.831±0.024
MIR	0.327	0.750±0.005	0.768±0.026	0.745±0.015	0.689±0.097	<u>0.788±0.022</u>	0.836±0.016
stkAAVE	0.426	0.702±0.042	0.368±0.011	0.397±0.022	0.743±0.006	0.650±0.028	<u>0.709±0.022</u>
ADX	0.362	<u>0.769±0.018</u>	0.723±0.002	0.718±0.004	0.784±0.002	0.673±0.022	0.679±0.024
Top rank ↑	0	2	3	0	8	1	<u>6</u>
Avg. rank ↓	6.60	3.60	4.30	4.30	<u>2.85</u>	3.70	2.40

in-degree, the weight of in-degree, out-degree, and weight of out-degree) and outputs binary classification prediction.

MiNT Models. We train six multi-network models, each with a different number of networks corresponding to 2^k datasets, where $k \in [1, 6]$. We name each multi-network model based on the number of datasets used in training; for example, MiNT-16 is trained with 16 datasets. The default TGNN architecture is HTGN, which shows superior performance, while GCLSTM architecture is also trained and discussed in Table 1.

Computational Resource. We ran all experiments on NVIDIA Quadro RTX 8000 (48G memory) with 4 standard CPU nodes (either Milan Zen 3 2.8 GHz and 768GB of memory each or Rome Zen 2, 2.5GHz and 256GB of memory each). Figure 6 shows the training time for MiNT models which scales linearly to number of training networks.

5.2. Experimental Results

The Benefit Of MiNT Training. We present the performance of our multi-network models trained with datasets of varying sizes and zero-shot inference tested on 20 unseen test datasets. The results are reported with an average and standard deviation of three runs with different random seeds.

We compare our results with five baseline single models: Persistence Forecast, GCLSTM, EvolveGCN, HTGN, and GraphPulse, which are trained and tested on the same 20 datasets. For visual clarity, Figure 4 shows the AUC on test data results for MiNT-4, MiNT-16 and MiNT-64 only as well as persistence forecasting and HTGN single model. We show the performance of all six multi-network models in Appendix Figure 9. Overall, an upward trend is observed in most datasets from multi-network models 2 to 64, such as in *BAG*, *MIR* and *BEPRO* datasets, highlighting the power of larger multi-network models in temporal graph learning. In Figure 4, the MiNT-64 yields the best AUC in 16 out of 20 test datasets. This result is significant because the multi-network models outperform the single models individually trained on these datasets. We detail the prediction performance of the models in Table 1, where we present the AUC values for both single-trained baselines and multi-network models, specifically MiNT-32 and MiNT-64, across various datasets. We also report the top rank, average rank, and win ratio for each model. The top rank indicates the number of datasets where a method ranks first. To calculate the average rank, we assign an AUC-based rank (ranging from 1 to 8) to every model across the 20 test datasets and compute the average. The win ratio represents the proportion of datasets where a model outperforms a single model.

Overall, MiNT-64 exhibits the best overall performance, achieving the state-of-the-art AUC performance in 6 net-

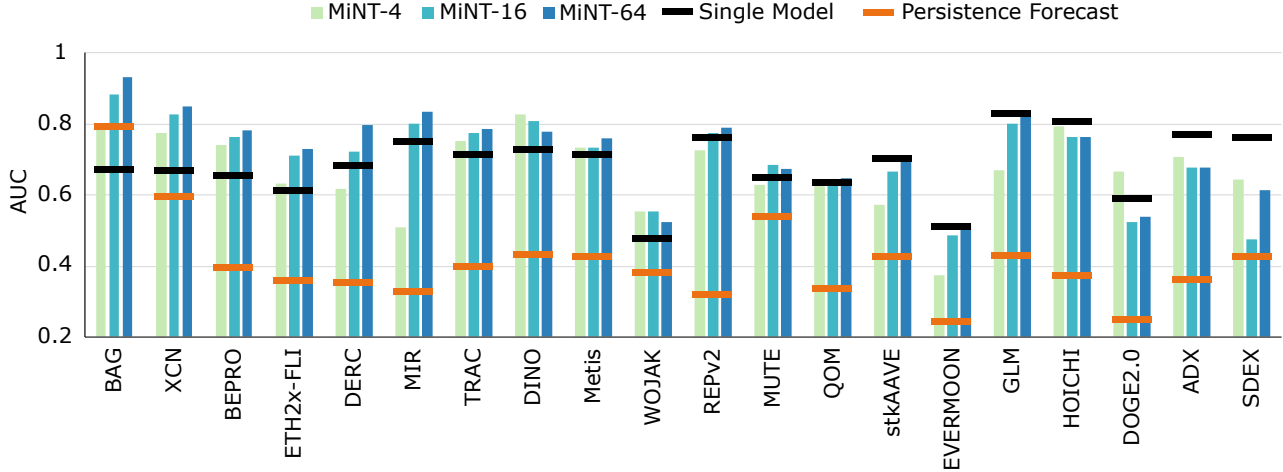


Figure 4: **MiNT Performance with Varying Training Scales.** Test AUC of MiNT models trained on 4, 16 and 64 networks and evaluated on unseen test datasets. We compare the performance with persistence forecast, and HTGN models trained and tested on each dataset.

works and top two performance in 14 out of 20 total test networks with zero-shot inference. Moreover, Appendix Table 9 indicates that the MiNT-64 also achieves superior performance on the test set of 38 out of 64 training networks when compared to the performance of single models. This demonstrates the strong generalizability and transferability of our MiNT-64 model. While GraphPulse achieves the highest top rank of 8, it relies on trained inference, unlike our multi-network models, which are based on zero-shot inference. Notably, training GraphPulse on each dataset is computationally expensive, while inference testing of our pre-trained MiNT-64 on all datasets takes only a few minutes. This makes the performance of MiNT-64 even more remarkable. Furthermore, despite trained models like HTGN or GCLSTM performing well on certain datasets, our MiNT-64 model consistently achieves competitive rankings across all datasets.

Table 2: Rank-based prediction performances.

Model	Top rank $\uparrow$	Avg. rank $\downarrow$	Win ratio $\uparrow$
Persist. forecast	0	7.9	0.00
Single model	3	4.35	-
MiNT-2	0	6.15	0.25
MiNT-4	2	4.35	0.45
MiNT-8	1	4.45	0.45
MiNT-16	1	3.45	0.65
MiNT-32	2	3.20	0.70
MiNT-64	11	2.15	0.80

Effect Of Scaling. In Table 2, we further compare the models by reporting the top rank, average rank, and win ratio for different configurations of the multi-network mod-

els. We observe a notable improvement in performance as the number of training networks increases. For instance, the average rank (lower is better) improves from 6.15 for MiNT-2 to 2.15 for MiNT-64, which signifies a roughly 50% performance enhancement when scaling from two networks to sixty-four. The improvement in the win ratio is also substantial, with MiNT-64 achieving the highest win ratio of 0.80, outperforming the other models in most datasets. This indicates that increasing the number of networks in a multi-network model significantly enhances its robustness and predictive power, particularly when compared to single models and smaller multi-network configurations. Overall, the Multi-network HTGN-based models have shown superior zero-shot performance and transferability ability.

MiNT Ablation Study. We conducted an ablation study for the MiNT-train algorithm to assess the effects of *context switching* and *order shuffling*. Models are trained the same as multi-network model training setup and tested on the 20 unseen test datasets. The average results are presented in Table 3. Training different multi-network models without resetting memory revealed that persistent memory across epochs negatively impacts generalization, emphasizing the importance of reset mechanisms to reduce overfitting. Additionally, we explored the necessity of shuffling data by fixing the order of training networks. The observed performance decline indicated that incorporating randomness to MiNT is vital for improving the model’s robustness and generalizability.

Effect Of Data Selection. We investigate the effect of data selection on the performance of MiNT models trained with different training data packs. As the first work on

Table 3: Ablation study results (AUC) demonstrating the impact of various training strategies on model performance.

Model	MiNT-4 $\uparrow$	MiNT-8 $\uparrow$	MiNT-16 $\uparrow$	MiNT-32 $\uparrow$	MiNT-64 $\uparrow$
Base Model	0.667 ± 0.111	0.676 ± 0.099	0.704 ± 0.115	0.714 ± 0.107	0.727 ± 0.114
w/o Order shuffling	0.647 ± 0.113	0.643 ± 0.117	0.690 ± 0.105	0.708 ± 0.099	0.694 ± 0.109
w/o Context Switching	0.667 ± 0.120	0.608 ± 0.102	0.693 ± 0.099	0.677 ± 0.098	0.688 ± 0.095

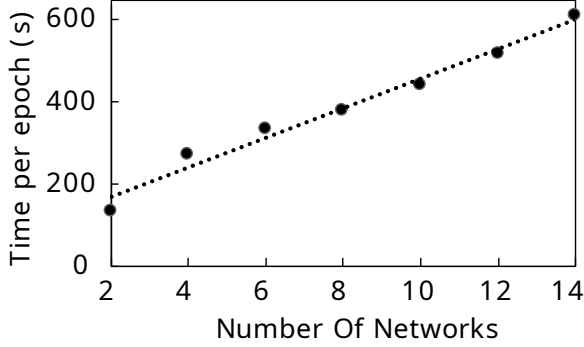


Figure 6: Time per epoch for training multi-network models.

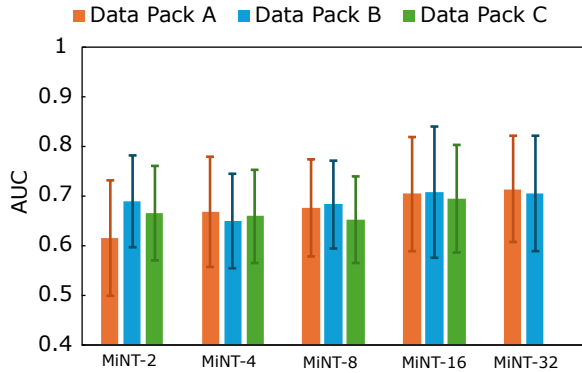


Figure 5: Effect of Data Selection on model performance.

multi-network training for temporal graphs, we explore the importance of our dataset selection process. To avoid any bias, we randomly sampled the training datasets from the 64 available networks. We conducted an empirical experiment to examine the impact of dataset selection on training MiNT models. In this experiment, we choose three disjoint sets of datasets (data pack A, B, and C) for training MiNT-2, MiNT-4, MiNT-8, and MiNT-16 and two disjoint sets of datasets (data pack A, B) for training MiNT-32. Using disjoint data packs ensures that each model is trained on unique data, eliminating any overlap that could obscure the results. We then test our models on 20 unseen test datasets.

As shown in Figures 5, as the number of training networks increases, the multi-network model performance increases while the variance between different choices of training

networks decreases. However, the difference between models that use the same number of datasets diminishes as we move from models of 2 to 32 datasets. We observe that smaller models (i.e., MiNT-2) have a higher variance when compared to larger models (i.e., MiNT-64); in addition, the model performance also increases from small to large models. For example, MiNT-64 outperforms MiNT-32 on 16 out of 20 datasets.

Time Complexity Analysis. The MiNT-train algorithm has the same complexity as training the single model across all the training networks. Specifically, for the best performing HTGN-based model, the time complexity using MiNT-train is $O(m \cdot (N_{max}dd' + d'|\mathcal{E}_{max}|))$ where m is the number of training networks, N_{max} is set to the maximum number of nodes of networks in the training set, d and d' are the dimensions of the input and output features while $|\mathcal{E}_{max}|$ is the maximum number of edges in a snapshot. Empirically, we observe that the MiNT-training time scales linearly to the number of networks as seen in Figure 6 where we report the time per epoch for each multi-network model.

6. Conclusion

In this work, we seek to address the question: given a collection of observed temporal graphs, can we predict the evolution of an unseen network within the same domain? We find that it is indeed possible to learn from temporal networks in the same domain and predict future trends for unseen networks. First, we collected and released a collection of 84 temporal networks for the temporal graph property prediction task. These datasets serve as the foundation for studying neural scaling laws and foundation models on temporal graphs. Next, to learn from a large number of temporal graphs, we present MiNT-train, the first algorithm for training TGNNs across multiple temporal networks. Experimentally, we show that the neural scaling law also applies to temporal graphs; in particular, the more training networks are used, the better the model performance on unseen test networks. In addition, our trained multi-network models can outperform single models trained on individual test networks. Our empirical observations show the high potential of training foundational models on temporal graphs. We believe our MiNT will pave the way for advancements in temporal graph foundation models, providing valuable resources that the community can utilize.

Broader Impacts

The proposed research on pre-training temporal graph neural networks across multiple networks has the potential to advance the field of machine learning and its applications significantly. By introducing methodologies to enhance the scalability and transferability of TGNNs, this work could revolutionize areas like network security, financial fraud detection, and real-time social network analysis, where dynamic and adaptive models are essential. The publicly available dataset of 84 Ethereum-based temporal networks will serve as a valuable resource for the research community, fostering innovation and collaboration. Furthermore, the principles of multi-network pre-training introduced here can inspire analogous advances in other temporal data domains, such as healthcare, transportation, and climate science. This research opens up a new direction in training generalizable temporal graph models that, for the first time, can be trained on distinct temporal networks, paving the way for Temporal Graph Foundation Models.

This work also introduces a set of Ethereum transaction token networks, which are publicly available to users who have the necessary resources, such as fast SSDs, large RAM, and ample disk space, to synchronize Ethereum clients and manually extract blocks. Additionally, all Ethereum data is accessible on numerous Ethereum explorer sites such as etherscan.io. An Ethereum user’s privacy depends on whether personally identifiable information (PII) is associated with any of their blockchain address, which serves as account handles and are considered pseudonymous. If such PII were obtained from other sources, our datasets could potentially be used to link Ethereum addresses. However, real-life identities can only be discovered using IP tracking information, which we neither have nor share. Our data does not contain any PII. Furthermore, we have developed a request to exclude an address from the dataset.

References

- Abnar, S., Dehghani, M., Neyshabur, B., and Sedghi, H. Exploring the limits of large scale pre-training. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=V3C8p78sDa>.
- Akcora, C. G., Li, Y., Gel, Y. R., and Kantarcioglu, M. Bitcoinheist: Topological data analysis for ransomware detection on the bitcoin blockchain. *CoRR*, abs/1906.07852, 2019. URL <http://arxiv.org/abs/1906.07852>.
- Awais, M., Naseer, M., Khan, S., Anwer, R. M., Cholakkal, H., Shah, M., Yang, M.-H., and Khan, F. S. Foundational models defining a new era in vision: A survey and outlook. *arXiv preprint arXiv:2307.13721*, 2023.
- Beaini, D., Huang, S., Cunha, J. A., Li, Z., Moisesescu-Pareja, G., Dymov, O., Maddrell-Mander, S., McLean, C., Wenkel, F., Müller, L., et al. Towards foundational models for molecular learning on large-scale multi-task datasets. In *The Twelfth International Conference on Learning Representations*, 2023.
- Brown, T. et al. Language models are few-shot learners. *NeurIPS*, 33:1877–1901, 2020.
- Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., Lee, P., Lee, Y. T., Li, Y., Lundberg, S., et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023.
- Chen, J., Wang, X., and Xu, X. GC-LSTM: graph convolution embedded LSTM for dynamic network link prediction. *Appl. Intell.*, 52(7):7513–7528, 2022. doi: 10.1007/S10489-021-02518-9. URL <https://doi.org/10.1007/s10489-021-02518-9>.
- Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., and Leskovec, J. Open graph benchmark: Datasets for machine learning on graphs. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *NeurIPS 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.
- Huang, S., Poursafaei, F., Danovitch, J., Fey, M., Hu, W., Rossi, E., Leskovec, J., Bronstein, M. M., Rabusseau, G., and Rabbany, R. Temporal graph benchmark for machine learning on temporal graphs. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *NeurIPS 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *CoRR*, abs/2001.08361, 2020. URL <https://arxiv.org/abs/2001.08361>.
- Kazemi, S. M., Goel, R., Jain, K., Kobayez, I., Sethi, A., Forsyth, P., and Poupart, P. Representation learning for dynamic graphs: A survey. *Journal of Machine Learning Research*, 21(70):1–73, 2020.
- Kim, H.-M., Bock, G.-W., and Lee, G. Predicting ethereum prices with machine learning based on blockchain information. *Expert Systems with Applications*, 184:115480, 2021.

- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In Bengio, Y. and LeCun, Y. (eds.), *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL <http://arxiv.org/abs/1412.6980>.
- Klaser, K., Banaszewski, B., Maddrell-Mander, S., McLean, C., Müller, L., Parviz, A., Huang, S., and Fitzgibbon, A. W. Minimol: A parameter-efficient foundation model for molecular learning. In *ICML 2024 Workshop on Efficient and Accessible Foundation Models for Biological Discovery*, 2024.
- Li, J., Shomer, H., Mao, H., Zeng, S., Ma, Y., Shah, N., Tang, J., and Yin, D. Evaluating graph neural networks for link prediction: Current pitfalls and new benchmarking. *NeurIPS*, 36, 2024.
- Liu, J., Mao, H., Chen, Z., Zhao, T., Shah, N., and Tang, J. Neural scaling laws on graphs. *CoRR*, abs/2402.02054, 2024. doi: 10.48550/ARXIV.2402.02054. URL <https://doi.org/10.48550/arXiv.2402.02054>.
- Longa, A., Lachi, V., Santin, G., Bianchini, M., Lepri, B., Liò, P., Scarselli, F., Passerini, A., et al. Graph neural networks for temporal graphs: State of the art, open challenges, and opportunities. *TRANSACTIONS ON MACHINE LEARNING RESEARCH*, 2023.
- Mao, H., Chen, Z., Tang, W., Zhao, J., Ma, Y., Zhao, T., Shah, N., Galkin, M., and Tang, J. Position: Graph foundation models are already here. In *ICML*, 2024.
- Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T. B., and Leiserson, C. E. Evolvegn: Evolving graph convolutional networks for dynamic graphs. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pp. 5363–5370. AAAI Press, 2020. doi: 10.1609/AAAI.V34I04.5984. URL <https://doi.org/10.1609/aaai.v34i04.5984>.
- Poursafaei, F., Huang, S., Pelrine, K., and Rabbany, R. Towards better evaluation for dynamic link prediction. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *NeurIPS 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- Radford, A. et al. Learning transferable visual models from natural language supervision. In *ICML*, pp. 8748–8763. PMLR, 2021.
- Rasul, K., Ashok, A., Williams, A. R., Ghonia, H., Bhagwatkar, R., Khorasani, A., Bayazi, M. J. D., Adamopoulos, G., Riachi, R., Hassen, N., Biloš, M., Garg, S., Schneider, A., Chapados, N., Drouin, A., Zantedeschi, V., Nevmyvaka, Y., and Rish, I. Lag-llama: Towards foundation models for probabilistic time series forecasting, 2024.
- Rosenfeld, J. S., Rosenfeld, A., Belinkov, Y., and Shavit, N. A constructive prediction of the generalization error across scales. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL <https://openreview.net/forum?id=ryenvpEKDr>.
- Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., and Bronstein, M. M. Temporal graph networks for deep learning on dynamic graphs. *CoRR*, abs/2006.10637, 2020. URL <https://arxiv.org/abs/2006.10637>.
- Salcedo-Sanz, S., Casillas-Pérez, D., Ser, J. D., Casanova-Mateo, C., Cuadra, L., Piles, M., and Camps-Valls, G. Persistence in complex systems. *Physics Reports*, 957: 1–73, 2022.
- Shamsi, K., Poursafaei, F., Huang, S., Ngo, B. T. G., Coskunuzer, B., and Akcora, C. G. Graphpulse: Topological representations for temporal graph property prediction. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=DZqic2sPTY>.
- Stojnic, V., Kalantidis, Y., and Tolias, G. Label propagation for zero-shot classification with vision-language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024*, pp. 23209–23218. IEEE, 2024. doi: 10.1109/CVPR52733.2024.02190. URL <https://doi.org/10.1109/CVPR52733.2024.02190>.
- Sypetkowski, M., Wenkel, F., Poursafaei, F., Dickson, N., Suri, K., Fradkin, P., and Beaini, D. On the scalability of gnns for molecular graphs. *arXiv preprint arXiv:2404.11568*, 2024.
- Wang, D., Zuo, Y., Li, F., and Wu, J. LLMs as zero-shot graph learners: Alignment of GNN representations with LLM token embeddings. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=32g9BWTndc>.

- Wang, Y., Chang, Y.-Y., Liu, Y., Leskovec, J., and Li, P. Inductive representation learning in temporal networks via causal anonymous walks. In *International Conference on Learning Representations (ICLR)*, 2021.
- Wood, G. et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.
- Xia, L. and Huang, C. Anygraph: Graph foundation model in the wild. *arXiv preprint arXiv:2408.10700*, 2024.
- Xia, L., Kao, B., and Huang, C. Opengraph: Towards open graph foundation models. *arXiv preprint arXiv:2403.01121*, 2024.
- Xu, D., Ruan, C., Körpeoglu, E., Kumar, S., and Achan, K. Inductive representation learning on temporal graphs. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL <https://openreview.net/forum?id=rJeWlyHYwH>.
- Yang, M., Zhou, M., Kalander, M., Huang, Z., and King, I. Discrete-time temporal network embedding via implicit hierarchical learning in hyperbolic space. In Zhu, F., Ooi, B. C., and Miao, C. (eds.), *KDD '21: The 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, Singapore, August 14-18, 2021*, pp. 1975–1985. ACM, 2021. doi: 10.1145/3447548.3467422. URL <https://doi.org/10.1145/3447548.3467422>.
- Zhang, Z., Luo, B., Lu, S., and He, B. Live graph lab: Towards open, dynamic and real transaction graphs with NFT. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *NeurIPS 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023.
- Zhao, J., Mostafa, H., Galkin, M., Bronstein, M., Zhu, Z., and Tang, J. Graphany: A foundation model for node classification on any graph. *arXiv preprint arXiv:2405.20445*, 2024.

Appendix

A. MiNT Datasets

Numerous graph benchmark datasets have been introduced to advance research within the temporal graph learning community. Poursafaei et al. (2022) introduced six dynamic graph datasets while proposing visualization techniques and novel negative edge sampling strategies to facilitate link prediction tasks of dynamic graphs. Following the good practice from OGB (Hu et al., 2020), (Huang et al., 2023) introduced TGB, which provides automated and reproducible results with a novel standardized evaluation pipeline for both link and node property prediction tasks. However, these datasets belong to different domains, making them unsuitable for studying the scaling laws of neural network models trained with a large number of datasets from the same domain. (Li et al., 2024) provide a temporal benchmark for evaluating graph neural networks in link prediction tasks, though their focus does not extend to training on multiple networks. Conversely, the Live Graph Lab dataset by (Zhang et al., 2023) offers a temporal dataset and benchmark, employed for tasks like temporal node classification using TGNNs. This work aims to explore multi-network training and understand the transferability across temporal graphs. Therefore, we curate a collection of temporal graphs rather than focusing on individual ones as in prior work.

A.1. Datasets Extraction

We utilize a dataset of temporal graphs sourced from the Ethereum blockchain (Wood et al., 2014). In this section, we will describe Ethereum, explain our data pipeline, and conclude by defining the characteristics of the resulting dataset.

Ethereum and ERC20 Token Networks. We create our transaction network data by first installing an Ethereum node and accessing the P2P network by using the Ethereum client Geth (<https://github.com/ethereum/go-ethereum>). Then, we use Ethereum-ETL (<https://github.com/blockchain-etl/ethereum-etl>) to parse all ERC20 tokens and extract asset transactions. We extracted more than sixty thousand ERC20 tokens from the entire history of the Ethereum blockchain. However, during the lifespans of most token networks, there are interim periods without any transactions. Additionally, a significant number of tokens live for only a short time span. To avoid training data quality challenges, we use 84 token networks with at least one transaction every day during their lifespan and are large enough to be used as a benchmark dataset for multi-network model training.

Temporal Networks. Each token network represents a dis-

tinct temporal graph, reflecting the time-stamped nature of its transactions. In these networks, nodes (addresses), edges (transactions), and edge weights (transaction values) evolve over time, capturing the dynamic behavior of the network. Additionally, these networks differ in their start dates and durations, introducing further variation in their evolution. While each token network operates independently with its own set of investors, they exhibit common patterns and behaviors characteristic of transaction networks. These similarities allow the model to learn and generalize from these patterns across different networks. Collecting temporal graphs from different ERC20 token networks allows for comparative analysis, uncovering in-common patterns and unique behaviors. This strengthens the model’s ability to generalize and improves its robustness.

Figure 7 illustrates the MiNT overview from dataset extraction and discretizing graph networks for model training step.

A.2. Dataset Statistics

Our MiNT dataset is a collection of 84 ERC20 token networks derived from Ethereum from 2017 to 2023. Each token network is represented as a dynamic graph, in which each address and transaction between addresses are a node and directed edge, respectively. The biggest MiNT token network contains 128,159 unique addresses and 554,705 transactions, while the smallest token network has 1,454 nodes.

Figure 3 shows that most networks have more than 10k nodes and over 100k edges. The lifespan of MiNT networks varies from 107 days to 6 years, and there exists at least one transaction each day. Figure 3.a shows the novelty scores, i.e., the average ratio of unseen edges in each timestamp, introduced by (Poursafaei et al., 2022). Figure 3 shows that most of the 84 networks have novelty scores greater than 0.3,

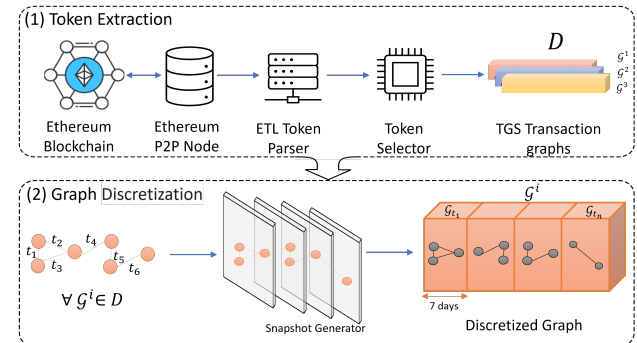


Figure 7: **MiNT data processing overview.** (1) *Token extraction*: extracting the token transaction network from the Ethereum node. (2) *Discretization*: creating weekly snapshots to form discrete time dynamic graphs.

Table 4: All token networks’ statistics.

Token	#Node	#Transaction	#Snapshots (days)	Growth rate	Novelty	Surprise	Token	#Node	#Transaction	#Snapshots (days)	Growth rate	Novelty	Surprise
ARC	11325	70968	606	0.43	0.32	0.88	Metis	52586	343141	907	0.44	0.48	0.89
CELR	65350	235807	1691	0.49	0.56	0.96	cDAI	52753	358050	1437	0.45	0.46	0.9
CMT	86895	205961	309	0.45	0.72	0.92	BITCOIN	34051	347054	178	0.48	0.39	0.63
DRGN	113453	341849	2164	0.44	0.57	0.97	INJ	60472	312822	1113	0.46	0.52	0.98
GHST	35156	180955	1146	0.43	0.51	0.93	MIM	23038	269366	885	0.44	0.4	0.89
INU	8556	66315	154	0.27	0.41	0.59	GLM	53385	234912	1080	0.5	0.53	0.96
IOTX	63079	288469	1993	0.45	0.56	0.99	Mog	14590	240680	107	0.37	0.38	0.55
QSP	117977	299671	2178	0.45	0.67	0.99	DPI	40627	234246	1150	0.49	0.5	0.86
REP	83282	224843	346	0.46	0.69	0.96	LINA	45342	227147	1144	0.45	0.46	0.95
RFD	23208	173695	169	0.3	0.39	0.6	YF-DAI	22466	226875	1158	0.42	0.31	0.87
TNT	88247	316352	1216	0.43	0.55	0.93	BOB	42806	212099	199	0.35	0.48	0.73
TRAC	71667	299181	2110	0.46	0.54	0.97	RGT	35277	211932	1110	0.44	0.46	0.98
RLB	28033	240291	129	0.43	0.49	0.76	TVK	42539	208082	1062	0.41	0.48	0.93
steCRV	19079	211538	1033	0.45	0.53	0.9	RSR	50645	205906	659	0.47	0.62	0.91
ALBT	63042	434881	1152	0.43	0.44	0.89	WOIAK	34341	198653	201	0.37	0.48	0.73
POLS	128159	554705	1132	0.45	0.61	0.94	ANT	36517	200262	1107	0.47	0.46	0.93
SWAP	69230	509769	1213	0.46	0.45	0.79	LADYS	37486	192176	181	0.37	0.52	0.79
SUPER	83299	502030	986	0.47	0.46	0.85	ETH2x-FLI	11008	199088	965	0.47	0.28	0.84
RARI	87186	502960	1207	0.43	0.47	0.91	TURBO	38638	189048	189	0.33	0.48	0.72
KP3R	39323	493258	1102	0.43	0.33	0.88	REPv2	39061	191367	1194	0.48	0.5	0.97
MIR	79984	444998	1066	0.45	0.43	0.92	NOIA	29798	185528	1133	0.46	0.37	0.7
aUSDC	23742	475680	1067	0.46	0.4	0.73	0x0	21531	182430	283	0.51	0.46	0.81
LUSD	25852	430473	943	0.48	0.36	0.87	PSYOP	25450	168896	169	0.32	0.39	0.59
PICKLE	28498	430262	1149	0.48	0.34	0.69	ShibDoge	40023	134697	680	0.43	0.53	0.8
DODO	47046	390443	1131	0.47	0.45	0.91	ADX	14567	123755	1188	0.44	0.4	0.91
YFII	43964	391984	1196	0.44	0.44	0.96	BAG	11860	122634	298	0.31	0.44	0.87
STARL	71590	369913	856	0.46	0.48	0.86	QOM	21757	118292	598	0.46	0.41	0.81
LQTY	34687	374230	943	0.45	0.34	0.91	BEPRO	26521	120261	1132	0.46	0.48	0.87
FEG	118294	367584	1007	0.4	0.62	0.92	AIOZ	29231	119926	947	0.43	0.49	0.89
AUDIO	91218	362685	1108	0.45	0.58	0.95	PRE	40476	118625	1113	0.5	0.55	0.86
OHM	45728	377068	690	0.43	0.46	0.88	CRU	19990	117712	1144	0.5	0.43	0.95
WOOL	16874	351178	716	0.41	0.18	0.41	POOH	27245	111641	193	0.26	0.49	0.69
DERC	24277	111205	824	0.45	0.49	0.83	aDAI	13648	187050	1068	0.45	0.46	0.82
stkAAVE	37355	110924	1128	0.42	0.57	0.71	ORN	44010	239451	1134	0.46	0.47	0.87
BTRFLY	8450	108371	453	0.48	0.34	0.44	DOGE2.0	7664	79047	123	0.45	0.38	0.66
SDEX	9127	104869	240	0.41	0.44	0.75	HOICHI	5075	77361	436	0.36	0.32	0.71
XCN	20085	104185	607	0.46	0.42	0.84	EVERMOON	7552	79868	163	0.24	0.35	0.52
HOP	37004	102650	514	0.41	0.6	0.88	MUTE	12426	82345	977	0.43	0.46	0.95
MAHA	18401	96180	749	0.43	0.47	0.91	crvUSD	2950	88647	174	0.61	0.37	0.73
DINO	15837	94140	358	0.44	0.44	0.74	SLP	6675	95368	1151	0.43	0.36	0.91
bendWETH	1454	96898	593	0.51	0.21	0.51	sILV2	12838	92905	611	0.4	0.34	0.48
PUSH	14501	93103	936	0.46	0.38	0.83	SPONGE	25852	90468	184	0.31	0.66	0.81

indicating that each day sees a considerable proportion of new edges in these token networks. We adopt a 70 – 15 – 15 split of train-test-validation for each token network and calculate the surprise score (Poursafaei et al., 2022), which indicates the number of edges that appear only in the test data. As Table 4 shows, the token networks have quite high surprise values with an average of 0.82. We also provide the node, edge and length distribution for train and test sets separately in Figure 8. Overall, train set datasets mostly have more nodes than those in the test set, while the number of edges and days are in the same range for both.

We summarize detailed statistics of each token network in MiNT datasets in Table 4. In the table, the growth rate is the ratio of label 1, indicating the increase in the number of edge counts concerning the problem definition defined in Appendix section C. In addition, we use the novelty and surprise scores introduced by Poursafaei et al. (2022). The novelty score is defined as the average ratio of new edges in each timestamp. The surprise score is defined as the ratio of edges that only appear in the test set. Formally,

$$novelty = \frac{1}{T} \sum_{t=1}^T \frac{|E^t \setminus E_{seen}^t|}{|E^t|}, \quad (1a)$$

$$surprise = \frac{|E_{test} \setminus E_{train}|}{|E_{test}|}, \quad (1g)$$

where E^t and E_{seen}^t denotes the set of edges present only in timestamp t and seen in previous timestamps, respectively. E_{test} represents edges that appear in the test set, and edges appearing in the train set are represented as E_{train} .

Comparison Between Training And Testing Set. Nodes, transactions, and length (in days) distribution over the training and testing sets are shown in Figure 8. Training sets well-support the multi-network model to generalize characteristics of the entire MiNT dataset due to the similarity between nodes, edge and length in days distributions shown in Figures 8(a), 8(b), 8(c) and those distributions across 84 token networks of MiNT datasets. In addition, the variance of datasets’ characteristics of the testing set is shown in Figures 8(d), 8(e) and 8(f).

Node Overlap Analysis. We analyze the overlap of nodes between different datasets and within each dataset, which helps demonstrate the highly dynamic nature of our datasets. Specifically, we compared the nodes in each test network with those in the training networks and calculated the average overlap. As shown in Table 5, on average, only 2% of the nodes are common between the training and test datasets, highlighting the rapidly changing structure of these networks. Furthermore, we analyzed the node overlap within each test dataset by splitting it into the standard train-validation-test setup. We compared the nodes in the

70% training snapshots with the nodes in the final 15% test snapshots, and on average, only 4% of the nodes overlapped. This indicates the highly inductive nature of our model and emphasizes the zero-shot challenge it addresses in this domain. These findings underscore the importance of tackling such dynamic and evolving challenges in temporal graph learning.

Table 5: Overlapping Nodes Statistics

Dataset	Average Node in Common vs Train Set of MiNT-64 ($\pm$ std)	Train vs Test Snapshots Node in Common
MIR	0.021 $\pm$ 0.019	0.007
DOGE2.0	0.026 $\pm$ 0.033	0.015
MUTE	0.033 $\pm$ 0.020	0.045
EVERMOON	0.023 $\pm$ 0.033	0.043
DERC	0.020 $\pm$ 0.020	0.031
ADX	0.024 $\pm$ 0.020	0.018
HOICHI	0.023 $\pm$ 0.013	0.053
SDEX	0.024 $\pm$ 0.019	0.141
BAG	0.019 $\pm$ 0.017	0.107
XCN	0.016 $\pm$ 0.010	0.034
ETH2x-FLI	0.038 $\pm$ 0.041	0.028
stkAAVE	0.026 $\pm$ 0.027	0.057
GLM	0.014 $\pm$ 0.015	0.047
QOM	0.018 $\pm$ 0.014	0.044
WOJAK	0.025 $\pm$ 0.032	0.018
DINO	0.018 $\pm$ 0.014	0.049
Metis	0.020 $\pm$ 0.013	0.041
REPV2	0.016 $\pm$ 0.017	0.013
TRAC	0.015 $\pm$ 0.016	0.031
BEPRO	0.023 $\pm$ 0.022	0.021

B. Temporal Graph Learning

In this section, we give further details about the temporal graph learning models we used as a baseline for our work.

Persistence Forecast uses data from the previous and current weeks to predict the next week’s property. If we observe an increasing trend in the number of transactions in the current week compared to the previous week, we predict a similar increasing trend for the following week. This simple model is based on the assumption that trends in transaction networks can persist over time. Our baseline method has three key aspects. First, we do not use any future information to generate the labels. Second, we compare the current week’s transaction count to that of the previous week to determine the trend. Finally, if the current week shows an increase, we predict the same trend for the next week. This straightforward approach provides a basic baseline for comparison against more sophisticated predictive models.

HTGN leverages the power of hyperbolic geometry, which is well-suited for capturing hierarchical structures and complex relationships in temporal networks. HTGN maps the temporal graph into hyperbolic space and utilizes hyperbolic graph neural networks and hyperbolic gated recurrent neural networks to model the evolving dynamics. It incorporates two key modules that are hyperbolic temporal contextual

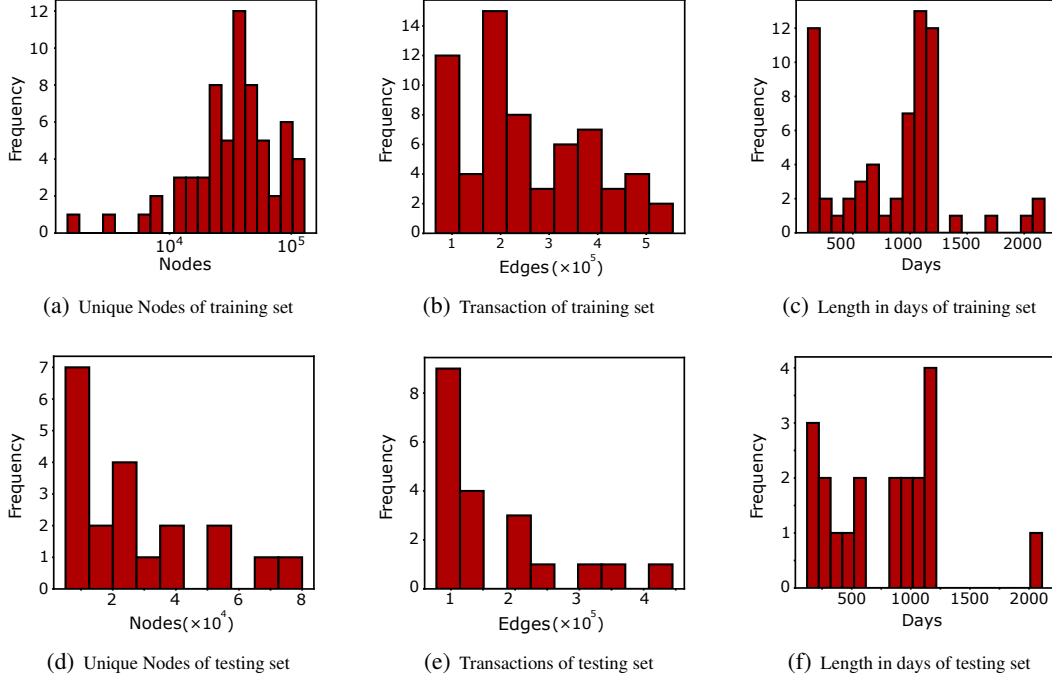


Figure 8: Distribution of the characteristics of the datasets over training and testing sets.

self-attention (HTA) and hyperbolic temporal consistency (HTC)-to ensure that temporal dependencies are effectively captured and that the model is both stable and generalizable across various tasks (Yang et al., 2021).

GraphPulse addresses the challenge of learning from nodes and edges with different timestamps, which many existing models struggle with. It combines two key techniques: the Mapper method from topological data analysis to extract clustering information from graph nodes and Recurrent Neural Networks (RNNs) for temporal reasoning. This principled approach helps capture both the structure and dynamics of evolving graphs (Shamsi et al., 2024).

GCLSTM combines a Graph Convolutional Network (GCN) and Long Short-Term Memory (LSTM) units to handle both the structural and temporal aspects of evolving networks. The GCN is used to capture the local structural properties of the network at each snapshot, while the LSTM learns the temporal evolution of these snapshots over time (Chen et al., 2022).

EvolveGCN is designed to capture the temporal dynamics of graph-structured data. Instead of relying on static node embeddings, EvolveGCN evolves the parameters of a graph convolutional network (GCN) over time. By using a recurrent neural network (RNN) to adapt the GCN parameters, this model is capable of dynamically adjusting during both training and testing, allowing it to handle evolving graphs, even when node sets vary significantly across different time

steps (Pareja et al., 2020).

C. Temporal Graph Property Prediction

In this study, we define graph property prediction as the task of predicting a specific graph property. In our case, this involves predicting the growth or shrinkage in the number of transactions in the next snapshot. Specifically, given the current weekly snapshot of a network, the objective is to predict the trend—whether the network will experience growth or shrinkage in transaction volume in the following week. This task has significant applications in the financial domain, as it provides insights into the willingness of investors to engage in a network and whether transaction activity is likely to increase. To ensure consistency, we use the same property prediction setting as GraphPulse (Shamsi et al., 2024), and the formal definition of the graph property is as follows:

Definition. We define network growth in terms of edge count as the predicted graph property. Let $\mathcal{G}$ represent a graph, t a specific time, δ_1 and δ_2 time intervals, and $E(t_1, t_n)$ the multi-set of edges between times t_1 and t_n . The property P is formally expressed as:

$$P(\mathcal{G}, t_1, t_n, \delta_1, \delta_2) = \begin{cases} 1, & \text{if } |E(t_n + \delta_1, t_n + \delta_2)| \\ & > |E(t_1, t_n)|, \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

Setting $n = 7$, $\delta_1 = 1$, and $\delta_2 = 7$, we establish a practical graph property with a 7-day prediction window. This choice is particularly relevant in financial contexts, such as Ethereum asset networks, where it can guide investment decisions, and in social network infrastructure, like Reddit, where it supports maintenance planning.

Insights For Transaction Networks. The graph growth/shrink property prediction in financial networks forecasts changes in transaction numbers (edge count), revealing trends in network activity. A growth in edge count indicates increased investor engagement, while a shrinkage suggests reduced activity or market hesitation. This property helps guide investment strategies, resource allocation, and risk management by providing insights into the evolving dynamics of transaction networks.

In temporal graphs, property predictions provide valuable insights into the dynamics and behaviors of evolving networks. While this work focuses on specific properties, numerous other characteristics can also be defined in this domain to highlight the significance of temporal graph property predictions. For instance, properties like the temporal global efficiency, temporal-correlation coefficient, and temporal betweenness centrality offer additional perspectives by capturing unique aspects of a graph’s temporal evolution. These examples further clarify the importance of studying temporal graph properties and their relevance to understanding complex network dynamics. Below, we formalize these three additional temporal graph properties and explain their relevance and insights which can be used in future works, particularly for transaction networks.

D. Hyperparameters

Single Models. We adopt a 70%–15%–15% split ratio for the train, validation, and test, respectively, for each token network, and during each epoch, the training model processes all snapshots in chronological order. We train every single model for a minimum of 100 and a maximum of 250 epochs with a learning rate set to 15×10^{-4} . We apply early stopping based on the AUC results on the validation set, with patience and tolerance set to 20 and 5×10^{-2} , respectively. Specifically, in HTGN training, the node embeddings are reset at the end of every epoch. We use Binary Cross-Entropy Loss for performance measurement and Adam (Kingma & Ba, 2015) as the optimization algorithm. It is important to note that the graph pooling layer, performance measurement, and optimization algorithm are also shared by the multi-network model training setup.

Multi-network Models. While following a similar training approach as in the single model training, we make specific adjustments for the multi-network model training. We set the number of epochs to 300 with a learning rate of 10^{-4}

and a train-validation-test chronological split ratio same as single models. Early stopping is applied based on the validation loss with a tolerance of 5×10^{-2} and the patience is set to 30. The best model is selected based on the validation AUC and used to predict the unseen test dataset.

E. Hyperbolic Temporal Graph Network

Hyperbolic geometry has been increasingly recognized for its ability to achieve state-of-the-art performance in several static graph embedding tasks (Yang et al., 2021). HTGN is a recent hyperbolic work that shows strong performance in learning over dynamic graphs in a DTDG manner. The model employs a hyperbolic graph neural network (HGNN) to learn the topological dependencies of the nodes and a hyperbolic-gated recurrent unit (HGRU) to capture the temporal dependencies. Temporal contextual attention (HTA) is also used To prevent recurrent neural networks from only emphasizing the most nearby time and to ensure stability along with generalization of the embedding. In addition, HTGN enables updating the model’s state at the test time to incorporate new information, which makes it a good candidate for learning the scaling law of TGNNs. In our MiNT framework, we use the HTGN architecture as part of our multi-network model because it excels in dynamic graph learning through hyperbolic geometry. Its strong performance makes it a valuable addition to our approach.

Given feature vectors X_t^E of snapshot t in Euclidean space, an HGNN layer first adopts an exponential map to project Euclidean space vectors to hyperbolic space as follows $X_t^H = \exp^c(X_t^E)$, and then performs aggregation and activation similar to GNN but in a hyperbolic manner, $\tilde{X}_t^H = \text{HGNN}(X_t^H)$. To prevent recurrent neural networks from only emphasizing the most nearby time and to ensure stability along with generalization of the embedding, HTGN uses temporal contextual attention (HTA) to generalize the latest w hidden states such that $\tilde{H}_{t-1}^H = \text{HTA}(H_{t-w}; \dots; H_{t-1})$ (Yang et al., 2021). HGRU takes the outputs from HGNN, $\tilde{X}_t^H$, and the attentive hidden state, $\tilde{H}_{t-1}^H$, from HTA as input to update gates and memory cells and then provides the latest hidden state as the output, $H_t^H = \text{HGRU}(\tilde{X}_t^H, \tilde{H}_{t-1}^H)$. To interpret hyperbolic embeddings, (Yang et al., 2021) adopt Poincaré ball model with negative curve $-c$, given $c > 0$, corresponds to the Riemannian manifold $(\mathbb{H}^{n,c}) = \{x \in \mathbb{R}^n : c\|x\|^2 < 1\}$ is an open n -dimensional ball. Given a Euclidean space vector $x_i^E \in \mathbb{R}^d$, we consider it as a point in tangent space $\mathcal{T}_{x'}\mathbb{H}^{d,c}$ and adopt the exponential map to project it into hyperbolic space :

$$x_i^H = \exp_{x'}^c(x_i^E) \quad (3)$$

Resulting in $x_i^H \in \mathbb{H}^{d,c}$, which is then served as input to the HGNN layer as follows (Yang et al., 2021):

$$\mathbf{m}_i^{\mathcal{H}} = W \otimes^c \mathbf{x}_i^{\mathcal{H}} \oplus^c \mathbf{b}, \quad (4a)$$

$$\tilde{\mathbf{m}}_i^{\mathcal{H}} = \exp_{\mathbf{x}'}^c \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \log_{\mathbf{x}'}^c(\mathbf{m}_j^{\mathcal{H}}) \right), \quad (4b)$$

$$\tilde{\mathbf{x}}_i^{\mathcal{H}} = \exp_{\mathbf{x}'}^c(\sigma(\log_{\mathbf{x}'}^c(\tilde{\mathbf{m}}_i^{\mathcal{H}}))). \quad (4c)$$

where W , b are learnable parameters and hyperbolic activation function σ achieved by applying logarithmic and exponential mapping. HGNN leverages attention-based aggregation by assigning attention score α_{ij} to indicate the importance of neighbour j to node i , computed as followed:

$$\alpha_{ij} = \text{softmax}_{(j \in \mathcal{N}(i))}(s_{ij}) = \frac{\exp(s_{ij})}{\sum_{j' \in \mathcal{N}(i)} \exp(s_{ij'})}, \quad (5)$$

$$s_{ij} = \text{LeakReLU}(a^T [\log_0^c(m_i^l) \parallel \log_0^c(m_j^l)]),$$

where a is trainable vector and $\parallel$ denotes concatenation operation.

The output of HGNN, $\tilde{X}_t^{\mathcal{H}}$, is then used as input to HGRU along with attentive hidden state $\tilde{H}_{t-1}^{\mathcal{H}}$ obtained by HTA, which generalize H_{t-1} to latest w snapshots $\{H_{t-w}, \dots, H_{t-1}\}$ (Yang et al., 2021). Operations behind HGRU are characterized by the following equation (Yang et al., 2021):

$$X_t^E = \log_{\mathbf{x}'}^c(\tilde{X}_t^{\mathcal{H}}), \quad (6a)$$

$$H_{t-1}^E = \log_{\mathbf{x}'}^c(\tilde{H}_{t-1}^{\mathcal{H}}), \quad (6b)$$

$$P_t^E = \sigma(W_z X_t^E + U_z H_{t-1}^E) \quad (6c)$$

$$R_t^E = \sigma(W_r X_t^E + U_r H_{t-1}^E), \quad (6d)$$

$$\tilde{H}_t^E = \tanh(W_h X_t^E + U_h(R_t \odot H_{t-1}^E)), \quad (6e)$$

$$H_t^E = (1 - P_t^E) \odot \tilde{H}_t^E + P_t^E \odot H_{t-1}^E, \quad (6f)$$

$$H_t^{\mathcal{H}} = \exp_{\mathbf{x}'}^c(H_t^E). \quad (6g)$$

where $W_z, W_r, W_h, U_z, U_r, U_h$ are the trainable weight matrices, P_t^E is the update gate to control the output and R_t^E is the reset gate to balance the input and memory (Yang et al., 2021).

F. Additional Results

Here, we present the test results for the six multi-network models trained on different network sizes, as well as the

single model and persistence forecast results. Figure 9 illustrates the AUC of these models on the test set. In most datasets, multi-network models outperform the single model, and in all datasets, they outperform the persistence forecast. We have also compared our model against additional state-of-the-art models, specifically including EvolveGCN (Pareja et al., 2020), GC-LSTM (Chen et al., 2022) and the only model designed for temporal graph properties prediction, GraphPulse (Shamsi et al., 2024) as baselines for the test set. In Table 6 and Table 7 the average and standard deviation of AUC and AP are presented respectively for all models. Surprisingly, MiNT-64 stands out as the best model, consistently achieving competitive performance in a greater number of datasets for both AUC and AP scores compared to all other models. Similarly, MN-32 shows strong performance, attaining the highest score in several datasets and placing second in numerous others, but it does not surpass MN-64 in overall rankings. These results show the power of multi-network models in performing downstream tasks on unseen datasets. Importantly, this high level of performance is achieved through zero-shot inference, meaning that the model was not specifically trained on these datasets. In contrast, other models, including GraphPulse, were trained directly for the datasets they evaluated. This considerable difference underscores the potential of MiNT-64 and highlights the power of zero-shot learning in effectively leveraging knowledge across different temporal graphs.

Table 8 presents the detailed performance of all MiNT models trained with GCLSTM. Notably, we observed a consistent trend with GCLSTM: as the model was trained on a larger number of networks, its zero-shot inference performance improved significantly. This highlights the positive impact of training on diverse networks for enhancing the model’s generalization capabilities.

We further compare the performance between MiNT-64 and single models on the training datasets shown in Table 9. There are 38 token networks MiNT-64 that outperform single models, while single models achieve better results in 26 datasets. In addition, the improvement in AUCs of MiNT-64 compared to single models across all training datasets is 2% on average, reflecting the improvement obtained from training multiple networks.

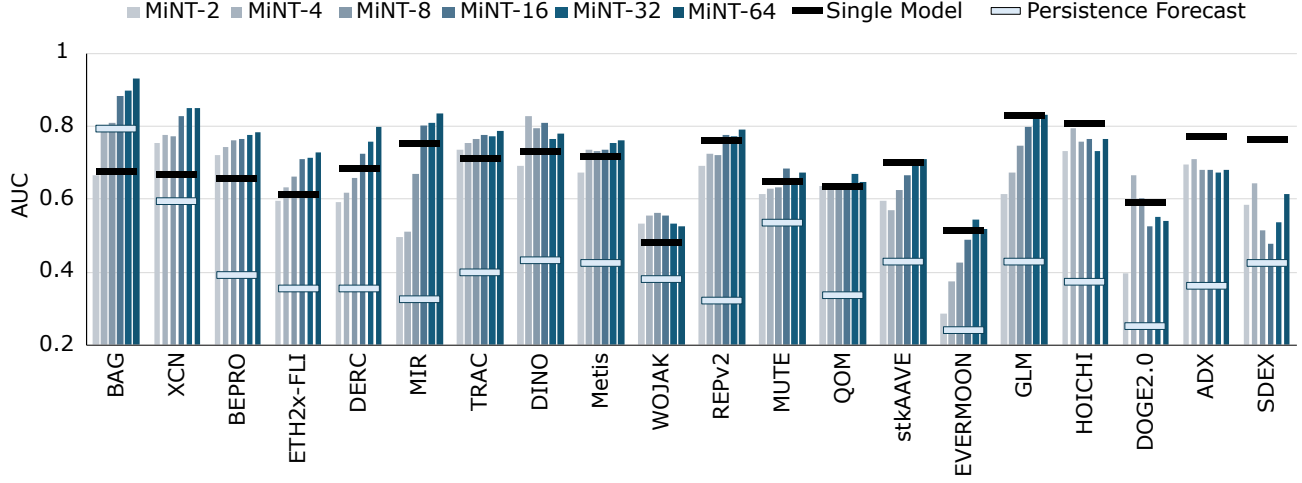


Figure 9: Test AUC of multi-network models trained on 2^n datasets where $n \in [1, 6]$ and evaluated on unseen test datasets. Comparing the performance with single models trained and tested on each dataset and persistence forecast results.

Table 6: AUC scores of multi-network models, single models, and persistence forecasts on test sets across three seeds, including comparisons with state-of-the-art models EvolveGCN, GC-LSTM and GraphPulse. The best performance is shown in bold, and the second best is underlined.

Token	Per. Fore.	GraphPulse	HTGN	GCLSTM	EvolveGCN	MiNT-2	MiNT-4	MiNT-8	MiNT-16	MiNT-32	MiNT-64
WOJAK	0.378	0.467 $\pm$ 0.030	0.479 $\pm$ 0.005	0.484 $\pm$ 0.000	0.505 $\pm$ 0.023	0.534 $\pm$ 0.020	<u>0.556 $\pm$ 0.029</u>	0.561 $\pm$ 0.018	0.556 $\pm$ 0.016	0.534 $\pm$ 0.017	0.524 $\pm$ 0.027
DOGE2.0	0.250	0.384 $\pm$ 0.18	0.590 $\pm$ 0.059	0.538 $\pm$ 0.000	0.551 $\pm$ 0.022	0.397 $\pm$ 0.124	0.667 $\pm$ 0.219	<u>0.603 $\pm$ 0.080</u>	0.526 $\pm$ 0.059	0.551 $\pm$ 0.022	0.538 $\pm$ 0.038
EVERMOON	0.241	0.519 $\pm$ 0.130	0.512 $\pm$ 0.023	0.562 $\pm$ 0.179	0.451 $\pm$ 0.046	0.287 $\pm$ 0.153	0.373 $\pm$ 0.037	0.426 $\pm$ 0.065	0.488 $\pm$ 0.054	<u>0.543 $\pm$ 0.075</u>	0.517 $\pm$ 0.039
QOM	0.334	0.775 $\pm$ 0.011	0.633 $\pm$ 0.017	0.612 $\pm$ 0.001	0.618 $\pm$ 0.002	0.635 $\pm$ 0.061	0.624 $\pm$ 0.025	0.633 $\pm$ 0.032	0.644 $\pm$ 0.009	<u>0.669 $\pm$ 0.034</u>	0.647 $\pm$ 0.019
SDEX	0.423	0.436 $\pm$ 0.030	0.762 $\pm$ 0.034	0.720 $\pm$ 0.002	<u>0.733 $\pm$ 0.028</u>	0.585 $\pm$ 0.139	0.643 $\pm$ 0.021	0.515 $\pm$ 0.031	0.476 $\pm$ 0.010	0.536 $\pm$ 0.042	0.614 $\pm$ 0.020
ETH2x-FLI	0.355	0.666 $\pm$ 0.047	0.610 $\pm$ 0.059	0.670 $\pm$ 0.009	0.688 $\pm$ 0.010	0.595 $\pm$ 0.083	0.632 $\pm$ 0.019	0.663 $\pm$ 0.018	0.710 $\pm$ 0.037	<u>0.715 $\pm$ 0.032</u>	0.729 $\pm$ 0.015
BEPRO	0.393	0.783 $\pm$ 0.003	0.655 $\pm$ 0.038	0.632 $\pm$ 0.019	0.610 $\pm$ 0.012	0.720 $\pm$ 0.028	0.742 $\pm$ 0.013	0.762 $\pm$ 0.007	0.765 $\pm$ 0.024	0.776 $\pm$ 0.008	<u>0.782 $\pm$ 0.003</u>
XCN	0.592	0.821 $\pm$ 0.004	0.668 $\pm$ 0.099	0.306 $\pm$ 0.092	0.512 $\pm$ 0.067	0.754 $\pm$ 0.025	0.774 $\pm$ 0.062	0.773 $\pm$ 0.076	0.827 $\pm$ 0.061	<u>0.848 $\pm$ 0.000</u>	0.851 $\pm$ 0.043
BAG	0.792	0.934 $\pm$ 0.020	0.673 $\pm$ 0.227	0.196 $\pm$ 0.179	0.329 $\pm$ 0.040	0.667 $\pm$ 0.134	0.802 $\pm$ 0.155	0.808 $\pm$ 0.095	0.884 $\pm$ 0.044	0.898 $\pm$ 0.075	<u>0.931 $\pm$ 0.028</u>
TRAC	0.400	0.767 $\pm$ 0.001	0.712 $\pm$ 0.071	0.748 $\pm$ 0.000	0.748 $\pm$ 0.000	0.734 $\pm$ 0.012	0.752 $\pm$ 0.009	0.764 $\pm$ 0.012	0.776 $\pm$ 0.012	<u>0.770 $\pm$ 0.007</u>	0.785 $\pm$ 0.008
DERC	0.353	0.769 $\pm$ 0.040	0.683 $\pm$ 0.013	0.703 $\pm$ 0.022	0.669 $\pm$ 0.009	0.593 $\pm$ 0.108	0.617 $\pm$ 0.030	0.657 $\pm$ 0.009	0.723 $\pm$ 0.058	0.756 $\pm$ 0.045	0.798 $\pm$ 0.027
Metis	0.423	0.812 $\pm$ 0.011	0.715 $\pm$ 0.122	0.646 $\pm$ 0.023	0.688 $\pm$ 0.027	0.672 $\pm$ 0.103	0.734 $\pm$ 0.017	0.730 $\pm$ 0.036	0.734 $\pm$ 0.016	0.753 $\pm$ 0.005	<u>0.760 $\pm$ 0.025</u>
REPv2	0.321	0.830 $\pm$ 0.001	0.760 $\pm$ 0.012	0.725 $\pm$ 0.014	0.709 $\pm$ 0.002	0.690 $\pm$ 0.024	0.725 $\pm$ 0.023	0.719 $\pm$ 0.022	0.774 $\pm$ 0.013	0.773 $\pm$ 0.013	<u>0.789 $\pm$ 0.020</u>
DINO	0.431	0.801 $\pm$ 0.020	0.730 $\pm$ 0.195	0.874 $\pm$ 0.028	<u>0.868 $\pm$ 0.029</u>	0.692 $\pm$ 0.140	0.827 $\pm$ 0.112	0.794 $\pm$ 0.096	0.809 $\pm$ 0.087	0.764 $\pm$ 0.048	0.779 $\pm$ 0.113
HOICHI	0.374	0.714 $\pm$ 0.010	0.807 $\pm$ 0.047	0.857 $\pm$ 0.000	<u>0.856 $\pm$ 0.001</u>	0.733 $\pm$ 0.101	0.795 $\pm$ 0.025	0.759 $\pm$ 0.040	0.763 $\pm$ 0.026	0.731 $\pm$ 0.029	0.765 $\pm$ 0.018
MUTE	0.536	0.779 $\pm$ 0.004	0.649 $\pm$ 0.015	0.593 $\pm$ 0.030	0.617 $\pm$ 0.010	0.613 $\pm$ 0.027	0.627 $\pm$ 0.024	0.633 $\pm$ 0.024	<u>0.684 $\pm$ 0.042</u>	0.657 $\pm$ 0.035	0.673 $\pm$ 0.013
GLM	0.427	0.769 $\pm$ 0.018	0.830 $\pm$ 0.029	0.451 $\pm$ 0.003	0.501 $\pm$ 0.033	0.613 $\pm$ 0.115	0.671 $\pm$ 0.034	0.746 $\pm$ 0.082	0.800 $\pm$ 0.062	0.826 $\pm$ 0.035	0.831 $\pm$ 0.024
MIR	0.327	0.689 $\pm$ 0.097	0.750 $\pm$ 0.005	0.768 $\pm$ 0.026	0.745 $\pm$ 0.015	0.497 $\pm$ 0.192	0.510 $\pm$ 0.015	0.669 $\pm$ 0.103	0.800 $\pm$ 0.044	<u>0.809 $\pm$ 0.022</u>	0.836 $\pm$ 0.016
stKAAVE	0.426	0.743 $\pm$ 0.006	0.702 $\pm$ 0.042	0.368 $\pm$ 0.011	0.397 $\pm$ 0.022	0.597 $\pm$ 0.076	0.571 $\pm$ 0.026	0.626 $\pm$ 0.023	0.666 $\pm$ 0.033	0.696 $\pm$ 0.027	<u>0.709 $\pm$ 0.022</u>
ADX	0.362	0.784 $\pm$ 0.002	0.769 $\pm$ 0.018	0.723 $\pm$ 0.002	0.718 $\pm$ 0.004	0.695 $\pm$ 0.003	0.708 $\pm$ 0.025	0.680 $\pm$ 0.008	0.678 $\pm$ 0.019	0.671 $\pm$ 0.015	0.679 $\pm$ 0.024

Table 7: **AP** scores of multi-network models, single models, and persistence forecasts on test sets across three seeds, including comparisons with state-of-the-art models EvolveGCN, GC-LSTM and GraphPulse. The best performance is shown in bold, and the second best is underlined.

Token	Per. Fore.	GraphPulse	HTGN	GCLSTM	EvolveGCN	MiNT-2	MiNT-4	MiNT-8	MiNT-16	MiNT-32	MiNT-64
WOJAK	0.658	0.863 $\pm$ 0.006	0.812 $\pm$ 0.003	0.812 $\pm$ 0.000	0.827 $\pm$ 0.017	0.832 $\pm$ 0.009	0.836 $\pm$ 0.015	0.842 $\pm$ 0.015	<u>0.850</u> $\pm$ 0.006	0.842 $\pm$ 0.008	0.837 $\pm$ 0.019
DOGE2.0	0.2	0.966 $\pm$ 0.002	0.933 $\pm$ 0.010	0.925 $\pm$ 0.000	0.927 $\pm$ 0.004	0.889 $\pm$ 0.031	<u>0.940</u> $\pm$ 0.050	0.936 $\pm$ 0.014	0.920 $\pm$ 0.014	0.927 $\pm$ 0.004	0.921 $\pm$ 0.014
EVERMOON	0.469	0.768 $\pm$ 0.01	0.585 $\pm$ 0.065	<u>0.612</u> $\pm$ 0.200	0.494 $\pm$ 0.017	0.442 $\pm$ 0.059	0.508 $\pm$ 0.045	0.542 $\pm$ 0.031	0.530 $\pm$ 0.040	0.567 $\pm$ 0.053	0.551 $\pm$ 0.021
QOM	0.315	0.840 $\pm$ 0.002	0.623 $\pm$ 0.024	<u>0.592</u> $\pm$ 0.001	0.597 $\pm$ 0.002	0.632 $\pm$ 0.070	0.617 $\pm$ 0.022	0.616 $\pm$ 0.007	0.626 $\pm$ 0.020	0.648 $\pm$ 0.027	<u>0.635</u> $\pm$ 0.027
SDEX	0.212	0.662 $\pm$ 0.017	0.825 $\pm$ 0.048	0.725 $\pm$ 0.002	<u>0.750</u> $\pm$ 0.025	0.723 $\pm$ 0.039	0.725 $\pm$ 0.021	0.650 $\pm$ 0.046	0.628 $\pm$ 0.036	0.697 $\pm$ 0.064	0.699 $\pm$ 0.021
ETH2x-FLI	0.381	0.836 $\pm$ 0.015	0.590 $\pm$ 0.103	0.735 $\pm$ 0.018	<u>0.756</u> $\pm$ 0.013	0.607 $\pm$ 0.122	0.621 $\pm$ 0.039	0.658 $\pm$ 0.057	0.745 $\pm$ 0.051	0.737 $\pm$ 0.049	<u>0.784</u> $\pm$ 0.007
BEPRO	0.374	0.802 $\pm$ 0.001	0.686 $\pm$ 0.042	0.637 $\pm$ 0.022	0.622 $\pm$ 0.009	0.743 $\pm$ 0.033	0.769 $\pm$ 0.015	0.799 $\pm$ 0.016	0.804 $\pm$ 0.034	<u>0.815</u> $\pm$ 0.007	0.816 $\pm$ 0.014
XCN	0.413	0.793 $\pm$ 0.002	0.687 $\pm$ 0.085	0.420 $\pm$ 0.032	0.555 $\pm$ 0.073	0.708 $\pm$ 0.065	0.765 $\pm$ 0.080	0.781 $\pm$ 0.082	0.829 $\pm$ 0.057	<u>0.851</u> $\pm$ 0.023	0.861 $\pm$ 0.042
BAG	0.504	0.957 $\pm$ 0.004	0.523 $\pm$ 0.290	0.235 $\pm$ 0.041	0.263 $\pm$ 0.011	0.474 $\pm$ 0.152	0.699 $\pm$ 0.193	0.682 $\pm$ 0.160	0.784 $\pm$ 0.118	0.829 $\pm$ 0.119	<u>0.889</u> $\pm$ 0.043
TRAC	0.4	0.767 $\pm$ 0.002	0.685 $\pm$ 0.074	0.716 $\pm$ 0.006	0.722 $\pm$ 0.001	0.705 $\pm$ 0.013	0.734 $\pm$ 0.012	0.741 $\pm$ 0.006	<u>0.764</u> $\pm$ 0.015	0.741 $\pm$ 0.015	0.758 $\pm$ 0.021
DERC	0.39	0.773 $\pm$ 0.004	0.532 $\pm$ 0.021	0.621 $\pm$ 0.053	0.513 $\pm$ 0.012	0.505 $\pm$ 0.157	0.477 $\pm$ 0.021	0.516 $\pm$ 0.030	0.639 $\pm$ 0.118	0.700 $\pm$ 0.080	<u>0.741</u> $\pm$ 0.024
Metis	0.38	0.801 $\pm$ 0.003	0.601 $\pm$ 0.187	0.575 $\pm$ 0.041	0.577 $\pm$ 0.006	0.532 $\pm$ 0.126	0.645 $\pm$ 0.029	0.632 $\pm$ 0.056	0.611 $\pm$ 0.021	<u>0.647</u> $\pm$ 0.026	0.639 $\pm$ 0.077
REPV2	0.376	0.797 $\pm$ 0.003	<u>0.758</u> $\pm$ 0.033	0.691 $\pm$ 0.006	0.689 $\pm$ 0.001	0.610 $\pm$ 0.063	0.619 $\pm$ 0.019	0.635 $\pm$ 0.042	0.705 $\pm$ 0.027	0.721 $\pm$ 0.004	0.729 $\pm$ 0.011
DINO	0.480	0.871 $\pm$ 0.026	0.747 $\pm$ 0.175	0.881 $\pm$ 0.029	<u>0.875</u> $\pm$ 0.024	0.738 $\pm$ 0.113	0.842 $\pm$ 0.102	0.793 $\pm$ 0.094	0.824 $\pm$ 0.077	0.753 $\pm$ 0.030	0.765 $\pm$ 0.119
HOICHI	0.602	0.623 $\pm$ 0.003	<u>0.666</u> $\pm$ 0.062	0.650 $\pm$ 0.000	0.658 $\pm$ 0.011	0.531 $\pm$ 0.109	0.677 $\pm$ 0.049	0.605 $\pm$ 0.037	0.609 $\pm$ 0.016	0.551 $\pm$ 0.045	0.594 $\pm$ 0.012
MUTE	0.38	0.726 $\pm$ 0.002	0.615 $\pm$ 0.049	0.504 $\pm$ 0.012	0.527 $\pm$ 0.015	0.579 $\pm$ 0.023	0.612 $\pm$ 0.041	0.603 $\pm$ 0.058	<u>0.675</u> $\pm$ 0.032	0.609 $\pm$ 0.021	0.647 $\pm$ 0.048
GLM	0.387	0.712 $\pm$ 0.047	0.797 $\pm$ 0.024	0.513 $\pm$ 0.001	0.529 $\pm$ 0.013	0.598 $\pm$ 0.121	0.651 $\pm$ 0.031	0.709 $\pm$ 0.088	0.783 $\pm$ 0.092	<u>0.819</u> $\pm$ 0.035	0.838 $\pm$ 0.032
MIR	0.405	0.766 $\pm$ 0.041	0.751 $\pm$ 0.003	0.765 $\pm$ 0.012	0.752 $\pm$ 0.007	0.493 $\pm$ 0.212	0.442 $\pm$ 0.024	0.645 $\pm$ 0.133	0.783 $\pm$ 0.064	<u>0.799</u> $\pm$ 0.015	0.811 $\pm$ 0.019
stkAAVE	0.207	0.751 $\pm$ 0.005	0.750 $\pm$ 0.020	0.506 $\pm$ 0.003	0.493 $\pm$ 0.009	0.662 $\pm$ 0.066	0.622 $\pm$ 0.011	0.694 $\pm$ 0.021	0.730 $\pm$ 0.037	0.741 $\pm$ 0.020	0.759 $\pm$ 0.019
ADX	0.372	0.765 $\pm$ 0.003	<u>0.758</u> $\pm$ 0.017	0.666 $\pm$ 0.002	0.661 $\pm$ 0.017	0.638 $\pm$ 0.021	0.667 $\pm$ 0.040	0.632 $\pm$ 0.010	0.621 $\pm$ 0.013	0.622 $\pm$ 0.018	0.628 $\pm$ 0.012

 Table 8: **AP and AUC** scores of GCLSTM-based multi-network models on test sets across three seeds. The best performance is shown in bold, and the second best is underlined.

Token	AUC						AP					
	MiNT-2	MiNT-4	MiNT-8	MiNT-16	MiNT-32	MiNT-64	MiNT-2	MiNT-4	MiNT-8	MiNT-16	MiNT-32	MiNT-64
MIR	0.653 $\pm$ 0.154	0.638 $\pm$ 0.090	0.588 $\pm$ 0.135	<u>0.765</u> $\pm$ 0.049	0.742 $\pm$ 0.036	0.789 $\pm$ 0.016	0.667 $\pm$ 0.153	0.602 $\pm$ 0.134	0.550 $\pm$ 0.166	0.750 $\pm$ 0.019	<u>0.758</u> $\pm$ 0.016	0.777 $\pm$ 0.013
DOGE2	0.487 $\pm$ 0.089	0.590 $\pm$ 0.146	0.487 $\pm$ 0.219	0.282 $\pm$ 0.097	0.769 $\pm$ 0.133	0.551 $\pm$ 0.022	0.910 $\pm$ 0.019	0.930 $\pm$ 0.030	0.907 $\pm$ 0.046	0.839 $\pm$ 0.057	0.965 $\pm$ 0.024	0.927 $\pm$ 0.004
MUTE	0.592 $\pm$ 0.076	0.627 $\pm$ 0.018	0.561 $\pm$ 0.035	0.589 $\pm$ 0.009	<u>0.627</u> $\pm$ 0.009	0.636 $\pm$ 0.003	0.534 $\pm$ 0.056	0.555 $\pm$ 0.017	0.502 $\pm$ 0.022	0.501 $\pm$ 0.006	<u>0.563</u> $\pm$ 0.002	0.568 $\pm$ 0.002
EVERMOON	0.429 $\pm$ 0.078	0.318 $\pm$ 0.152	0.306 $\pm$ 0.085	0.315 $\pm$ 0.154	<u>0.420</u> $\pm$ 0.084	0.494 $\pm$ 0.048	0.493 $\pm$ 0.095	0.423 $\pm$ 0.097	0.427 $\pm$ 0.037	0.447 $\pm$ 0.123	<u>0.530</u> $\pm$ 0.048	0.560 $\pm$ 0.010
DERC	0.614 $\pm$ 0.129	0.618 $\pm$ 0.058	0.569 $\pm$ 0.085	0.736 $\pm$ 0.027	0.647 $\pm$ 0.054	<u>0.696</u> $\pm$ 0.011	0.541 $\pm$ 0.150	0.546 $\pm$ 0.113	0.460 $\pm$ 0.078	0.693 $\pm$ 0.032	0.559 $\pm$ 0.087	<u>0.629</u> $\pm$ 0.012
ADX	0.692 $\pm$ 0.007	0.605 $\pm$ 0.182	0.674 $\pm$ 0.008	0.676 $\pm$ 0.003	<u>0.678</u> $\pm$ 0.004	0.674 $\pm$ 0.022	0.614 $\pm$ 0.011	0.583 $\pm$ 0.147	0.634 $\pm$ 0.024	0.609 $\pm$ 0.005	<u>0.617</u> $\pm$ 0.004	0.611 $\pm$ 0.010
HOICHI	0.663 $\pm$ 0.312	0.793 $\pm$ 0.065	0.633 $\pm$ 0.197	<u>0.817</u> $\pm$ 0.010	0.816 $\pm$ 0.043	0.847 $\pm$ 0.005	0.529 $\pm$ 0.240	0.602 $\pm$ 0.066	0.471 $\pm$ 0.178	<u>0.637</u> $\pm$ 0.016	0.630 $\pm$ 0.055	0.656 $\pm$ 0.014
SDEX	0.619 $\pm$ 0.210	0.721 $\pm$ 0.032	0.574 $\pm$ 0.233	0.741 $\pm$ 0.014	0.717 $\pm$ 0.020	<u>0.724</u> $\pm$ 0.002	0.678 $\pm$ 0.115	<u>0.732</u> $\pm$ 0.024	0.670 $\pm$ 0.092	0.752 $\pm$ 0.007	0.728 $\pm$ 0.009	0.729 $\pm$ 0.002
BAG	0.573 $\pm$ 0.072	0.525 $\pm$ 0.010	0.374 $\pm$ 0.029	0.442 $\pm$ 0.039	0.469 $\pm$ 0.060	<u>0.529</u> $\pm$ 0.023	0.358 $\pm$ 0.036	0.334 $\pm$ 0.005	0.277 $\pm$ 0.010	0.303 $\pm$ 0.013	0.311 $\pm$ 0.025	<u>0.337</u> $\pm$ 0.009
XCN	0.753 $\pm$ 0.026	<u>0.739</u> $\pm$ 0.005	0.726 $\pm$ 0.014	0.736 $\pm$ 0.006	0.731 $\pm$ 0.005	0.733 $\pm$ 0.003	0.690 $\pm$ 0.064	0.657 $\pm$ 0.009	<u>0.665</u> $\pm$ 0.031	0.656 $\pm$ 0.007	0.650 $\pm$ 0.003	0.653 $\pm$ 0.002
ETH2x-FLI	0.621 $\pm$ 0.119	0.615 $\pm$ 0.074	0.542 $\pm$ 0.086	<u>0.675</u> $\pm$ 0.008	0.666 $\pm$ 0.021	0.697 $\pm$ 0.010	0.669 $\pm$ 0.165	0.669 $\pm$ 0.084	0.570 $\pm$ 0.154	<u>0.752</u> $\pm$ 0.015	0.747 $\pm$ 0.021	0.766 $\pm$ 0.006
stkAAVE	0.601 $\pm$ 0.121	0.573 $\pm$ 0.084	0.517 $\pm$ 0.071	0.609 $\pm$ 0.032	<u>0.624</u> $\pm$ 0.017	0.650 $\pm$ 0.028	0.687 $\pm$ 0.101	0.616 $\pm$ 0.108	0.571 $\pm$ 0.045	0.669 $\pm$ 0.066	0.710 $\pm$ 0.017	0.736 $\pm$ 0.022
GLM	0.448 $\pm$ 0.097	0.363 $\pm$ 0.132	0.331 $\pm$ 0.083	0.563 $\pm$ 0.016	0.463 $\pm$ 0.053	0.502 $\pm$ 0.027	0.467 $\pm$ 0.041	0.436 $\pm$ 0.052	0.437 $\pm$ 0.020	0.541 $\pm$ 0.010	0.480 $\pm$ 0.026	<u>0.490</u> $\pm$ 0.012
QOM	0.594 $\pm$ 0.043	0.613 $\pm$ 0.009	0.574 $\pm$ 0.030	<u>0.614</u> $\pm$ 0.005	0.614 $\pm$ 0.007	0.618 $\pm$ 0.004	0.587 $\pm$ 0.020	<u>0.598</u> $\pm$ 0.004	0.573 $\pm$ 0.018	0.596 $\pm$ 0.005	0.597 $\pm$ 0.005	0.599 $\pm$ 0.003
WOJAK	0.516 $\pm$ 0.057	0.524 $\pm$ 0.016	<u>0.561</u> $\pm$ 0.026	0.489 $\pm$ 0.060	0.598 $\pm$ 0.075	0.534 $\pm$ 0.020	0.810 $\pm$ 0.052	0.838 $\pm$ 0.014	0.834 $\pm$ 0.016	0.808 $\pm$ 0.027	0.862 $\pm$ 0.026	<u>0.844</u> $\pm$ 0.007
DINO	0.667 $\pm$ 0.138	0.695 $\pm$ 0.147	0.738 $\pm$ 0.047	0.617 $\pm$ 0.148	<u>0.704</u> $\pm$ 0.065	0.659 $\pm$ 0.039	0.719 $\pm$ 0.120	<u>0.740</u> $\pm$ 0.107	0.746 $\pm$ 0.083	0.619 $\pm$ 0.073	0.683 $\pm$ 0.046	0.643 $\pm$ 0.041
Metis	<u>0.692</u> $\pm$ 0.023	0.677 $\pm$ 0.030	0.609 $\pm$ 0.025	0.674 $\pm$ 0.020	0.690 $\pm$ 0.016	0.697 $\pm$ 0.013	0.558 $\pm$ 0.029	0.541 $\pm$ 0.083	0.485 $\pm$ 0.065	0.555 $\pm$ 0.019	0.586 $\pm$ 0.022	<u>0.564</u> $\pm$ 0.019
REPV2	0.670 $\pm$ 0.053	0.686 $\pm$ 0.043	0.706 $\pm$ 0.040	0.735 $\pm$ 0.017	0.707 $\pm$ 0.019	<u>0.733</u> $\pm$ 0.019	0.617 $\pm$ 0.080	0.633 $\pm$ 0.008	0.619 $\pm$ 0.054	0.720 $\pm$ 0.031	0.654 $\pm$ 0.032	<u>0.683</u> $\pm$ 0.022
TRAC	0.736 $\pm$ 0.015	0.736 $\pm$ 0.014	0.710 $\pm$ 0.027	0.741 $\pm$ 0.003	0.741 $\pm$ 0.007	0.742 $\pm$ 0.004	0.702 $\pm$ 0.031	0.709 $\pm$ 0.020	0.708 $\pm$ 0.016	0.720 $\pm$ 0.002	0.717 $\pm$ 0.003	0.717 $\pm$ 0.005
BEPRO	0.723 $\pm$ 0.053	0.720 $\pm$ 0.035	0.685 $\pm$ 0.069	0.734 $\pm$ 0.016	0.757 $\pm$ 0.015	<u>0.746</u> $\pm$ 0.015	0.730 $\pm$ 0.096	0.755 $\pm$ 0.019	0.727 $\pm$ 0.063	0.764 $\pm$ 0.021	0.791 $\pm$ 0.007	<u>0.776</u> $\pm$ 0.012

Table 9: AUC scores of multi-network models, single models, and persistence forecasts on train sets across three seeds. The best performance is shown in bold.

Token	Per. Fore.	Single Model	MiNT-64	Token	Per. Fore.	Single Model	MiNT-64
LADYS	0.324	0.359 $\pm$ 0.127	0.744 $\pm$ 0.020	PUSH	0.450	0.635 $\pm$ 0.010	0.657 $\pm$ 0.027
Mog	0.333	0.426 $\pm$ 0.042	0.708 $\pm$ 0.066	CMT	0.262	0.726 $\pm$ 0.095	0.737 $\pm$ 0.031
STARL	0.219	0.444 $\pm$ 0.005	0.677 $\pm$ 0.023	SLP	0.415	0.500 $\pm$ 0.015	0.510 $\pm$ 0.022
LQTY	0.366	0.533 $\pm$ 0.218	0.729 $\pm$ 0.009	LUSD	0.372	0.722 $\pm$ 0.013	0.729 $\pm$ 0.08
WOOL	0.507	0.607 $\pm$ 0.063	0.776 $\pm$ 0.058	LINA	0.428	0.749 $\pm$ 0.052	0.755 $\pm$ 0.005
BITCOIN	0.382	0.586 $\pm$ 0.040	0.744 $\pm$ 0.042	AIOZ	0.39	0.745 $\pm$ 0.013	0.745 $\pm$ 0.012
aDAI	0.434	0.528 $\pm$ 0.033	0.657 $\pm$ 0.045	sILV2	0.581	0.886 $\pm$ 0.001	0.602 $\pm$ 0.023
RSR	0.542	0.530 $\pm$ 0.016	0.649 $\pm$ 0.028	INU	0.292	1.000 $\pm$ 0.000	0.793 $\pm$ 0.167
DRGN	0.385	0.561 $\pm$ 0.095	0.660 $\pm$ 0.005	SPONGE	0.167	0.650 $\pm$ 0.088	0.458 $\pm$ 0.027
FEG	0.442	0.500 $\pm$ 0.025	0.593 $\pm$ 0.011	bendWETH	0.49	0.651 $\pm$ 0.047	0.522 $\pm$ 0.019
SWAP	0.468	0.633 $\pm$ 0.013	0.724 $\pm$ 0.005	TNT	0.469	0.796 $\pm$ 0.006	0.700 $\pm$ 0.027
POLS	0.393	0.736 $\pm$ 0.022	0.818 $\pm$ 0.007	CRU	0.431	0.793 $\pm$ 0.053	0.698 $\pm$ 0.007
ORN	0.333	0.669 $\pm$ 0.08	0.747 $\pm$ 0.009	AUDIO	0.441	0.789 $\pm$ 0.023	0.694 $\pm$ 0.006
HOP	0.415	0.664 $\pm$ 0.170	0.738 $\pm$ 0.021	ALBT	0.317	0.734 $\pm$ 0.01	0.642 $\pm$ 0.009
RFD	0.277	0.721 $\pm$ 0.021	0.794 $\pm$ 0.021	MAHA	0.284	0.902 $\pm$ 0.024	0.815 $\pm$ 0.031
aUSDC	0.513	0.685 $\pm$ 0.009	0.757 $\pm$ 0.016	YFII	0.315	0.775 $\pm$ 0.039	0.691 $\pm$ 0.024
steCRV	0.360	0.591 $\pm$ 0.035	0.660 $\pm$ 0.006	BTRFLY	0.127	0.846 $\pm$ 0.029	0.765 $\pm$ 0.039
ANT	0.469	0.629 $\pm$ 0.039	0.695 $\pm$ 0.010	RLB	0.273	0.963 $\pm$ 0.032	0.887 $\pm$ 0.042
SUPER	0.432	0.729 $\pm$ 0.010	0.794 $\pm$ 0.000	crvUSD	0.291	0.417 $\pm$ 0.126	0.344 $\pm$ 0.025
ShibDoge	0.514	0.768 $\pm$ 0.034	0.832 $\pm$ 0.006	KP3R	0.528	0.839 $\pm$ 0.033	0.775 $\pm$ 0.016
0x0	0.383	0.555 $\pm$ 0.158	0.616 $\pm$ 0.018	CELR	0.495	0.782 $\pm$ 0.011	0.722 $\pm$ 0.032
NOIA	0.359	0.567 $\pm$ 0.042	0.626 $\pm$ 0.003	INJ	0.444	0.741 $\pm$ 0.093	0.686 $\pm$ 0.034
REP	0.360	0.711 $\pm$ 0.116	0.769 $\pm$ 0.019	QSP	0.431	0.695 $\pm$ 0.006	0.647 $\pm$ 0.006
DODO	0.346	0.717 $\pm$ 0.022	0.774 $\pm$ 0.013	RGT	0.396	0.864 $\pm$ 0.056	0.826 $\pm$ 0.007
PSYOP	0.403	0.848 $\pm$ 0.034	0.904 $\pm$ 0.018	ARC	0.532	0.760 $\pm$ 0.004	0.736 $\pm$ 0.020
TURBO	0.789	0.840 $\pm$ 0.012	0.893 $\pm$ 0.018	BOB	0.105	0.717 $\pm$ 0.026	0.696 $\pm$ 0.053
Yf-DAI	0.434	0.725 $\pm$ 0.006	0.777 $\pm$ 0.002	GHST	0.344	0.734 $\pm$ 0.050	0.726 $\pm$ 0.015
cDAI	0.519	0.566 $\pm$ 0.229	0.606 $\pm$ 0.014	PICKLE	0.321	0.838 $\pm$ 0.031	0.831 $\pm$ 0.011
POOH	0.250	0.755 $\pm$ 0.266	0.786 $\pm$ 0.019	IOTX	0.366	0.706 $\pm$ 0.039	0.699 $\pm$ 0.019
PRE	0.481	0.673 $\pm$ 0.024	0.702 $\pm$ 0.010	RARI	0.440	0.750 $\pm$ 0.019	0.744 $\pm$ 0.011
TVK	0.376	0.711 $\pm$ 0.014	0.738 $\pm$ 0.005	DPI	0.291	0.751 $\pm$ 0.014	0.746 $\pm$ 0.012
OHM	0.652	0.617 $\pm$ 0.004	0.643 $\pm$ 0.011	MIM	0.372	0.698 $\pm$ 0.010	0.697 $\pm$ 0.015